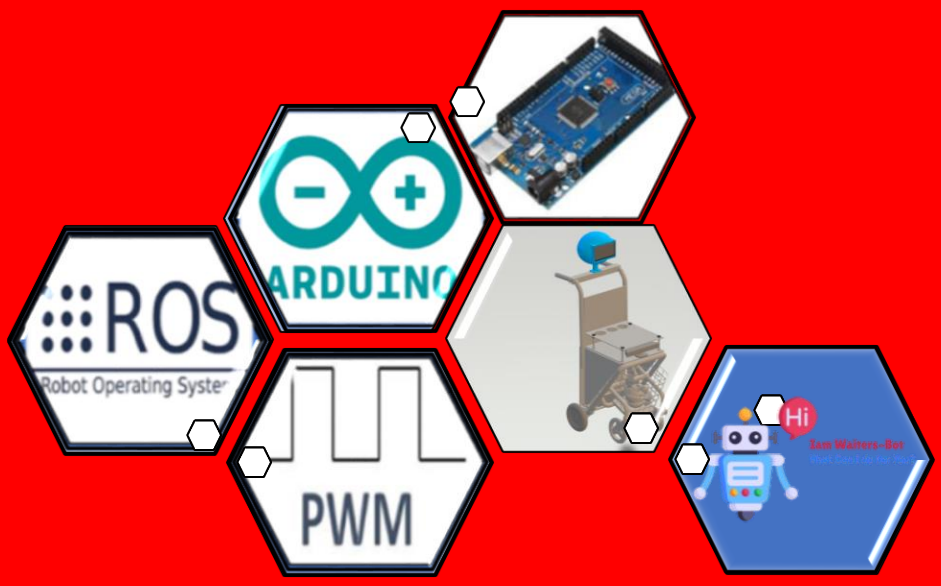
PENINGKATAN KUALITAS GERAK DAN GRAPHICAL USER INTERFACE MULTI-MODE PADA ROBOT PELAYAN Y-BOT BERBASIS ROBOT OPERATING SYSTEM



MUHAMMAD HUSEIN FHADELLAH

D041211093

PROGRAM STUDI SARJANA TEKNIK ELEKTRO

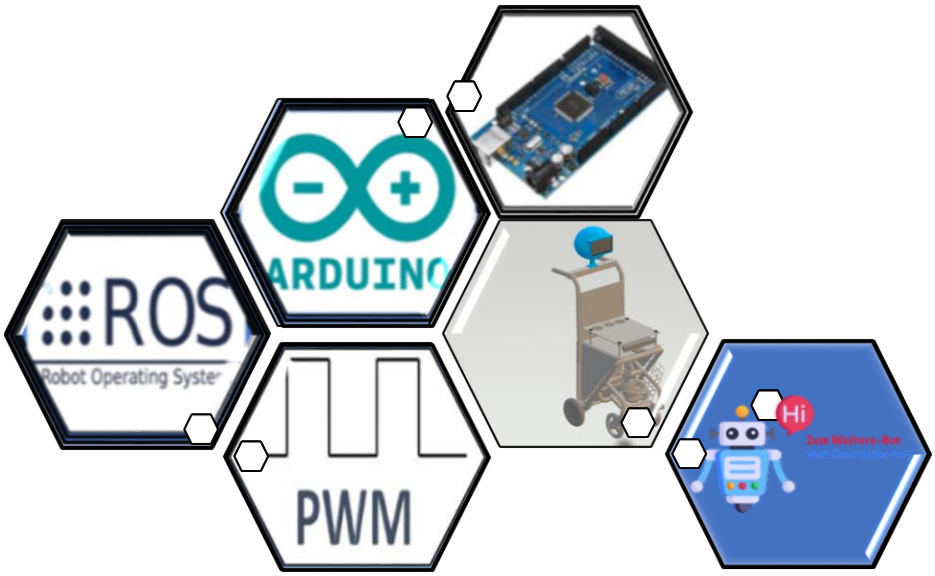
FAKULTAS TEKNIK

UNIVERSITAS HASANUDDIN

GOWA

2026

PENINGKATAN KUALITAS GERAK DAN GRAPHICAL USER INTERFACE MULTI-MODE PADA ROBOT PELAYAN Y-BOT BERBASIS ROBOT OPERATING SYSTEM



MUHAMMAD HUSEIN FHADELLAH

D041211093

PROGRAM STUDI SARJANA TEKNIK ELEKTRO

FAKULTAS TEKNIK

UNIVERSITAS HASANUDDIN

GOWA

2026

PENGAJUAN SKRIPSI

PENINGKATAN KUALITAS GERAK DAN GRAPHICAL USER INTERFACE MULTI-MODE PADA ROBOT PELAYAN Y-BOT BERBASIS ROBOT OPERATING SYSTEM

MUHAMMAD HUSEIN FHADELLAH

D041211093

Skripsi

Sebagai Salah Satu Syarat untuk Mencapai Gelar Sarjana

Program Studi Teknik Elektro

Pada

PROGRAM STUDI SARJANA TEKNIK ELEKTRO

FAKULTAS TEKNIK

UNIVERSITAS HASANUDDIN

GOWA

2026

ABSTRAK

MUHAMMAD HUSEIN FHADELLAH. **Peningkatan Kualitas Gerak dan Graphical User Interface Multi-Mode Pada Robot Pelayan Y-Bot Berbasis Robot Operating System** (dibimbing oleh Prof. Ir. Muh. Anshar, S.T., M.Sc-Res, Ph.D, IPU).

Robot pelayan atau Waiter-Bot adalah robot yang memerlukan kemampuan navigasi otonom yang handal dan interaksi pengguna yang intuitif pada lingkungan dinamis. Pada penelitian sebelumnya, ditemukan kendala mekanis berupa pergerakan kasar akibat sinyal kontrol motor instan yang menyebabkan slip roda, akumulasi drift odometri, dan distorsi pada peta navigasi. Penelitian ini bertujuan untuk meningkatkan kualitas gerak robot menggunakan algoritma kontrol Pulse Width Modulation (PWM) Ramp Rate dan mengembangkan Graphical User Interface (GUI) berbasis Kivy yang terintegrasi dengan Robot Operating System (ROS). Metode penelitian dilakukan dengan membandingkan performa sistem kontrol motor konvensional (instan) dengan metode Ramp Rate melalui pengujian kecepatan linear, angular, akurasi pemetaan (mapping), dan performa navigasi. Hasil pengujian menunjukkan bahwa penerapan PWM Ramp Rate mampu menghasilkan pergerakan yang lebih halus dan menghilangkan sentakan pada rentang kecepatan 0.5 m/s hingga 1.5 m/s. Kualitas pemetaan mengalami peningkatan signifikan, ditandai dengan penurunan rata-rata error dimensi peta dari 15.29% pada metode sebelumnya menjadi 6.18% menggunakan metode yang diusulkan. Selain itu, pada uji navigasi dengan target kecepatan 1 m/s, robot mampu mencapai kecepatan aktual rata-rata 1 m/s dengan waktu tempuh yang lebih efisien dibandingkan kontrol sebelumnya. Integrasi GUI Kivy berhasil memfasilitasi pengendalian multi-mode operasi (manual, mapping, navigasi) secara real-time dan efisien.

Kata Kunci: Gui Kivy, PWM Ramp Rate, ROS, Waiter-Bot.

ABSTRACT

Muhammad Husein Fhadellah. **Improving Quality Motion and Graphical User Interface On Waiter Robot Y-Bot Based Robot Operating System** (guided by Prof. Ir. Muh. Anshar, S.T., M.Sc-Res, Ph.D, IPU)

Waiter-Bots are robots that require reliable autonomous navigation capabilities and intuitive user interaction in dynamic environments. In previous studies, mechanical constraints were found in the form of rough movements due to instant motor control signals that caused wheel slip, odometry drift accumulation, and distortion in the navigation map. This study aims to improve the quality of robot movement using the Pulse Width Modulation (PWM) Ramp Rate control algorithm and to develop a Kivy-based Graphical User Interface (GUI) integrated with the Robot Operating System (ROS). The research method was carried out by comparing the performance of the conventional (instant) motor control system with the Ramp Rate method through testing linear speed, angular speed, mapping accuracy, and navigation performance. The test results show that the application of PWM Ramp Rate is capable of producing smoother movements and eliminating jerks in the speed range of 0.5 m/s to 1.5 m/s. The mapping quality has improved significantly, marked by a decrease in the average map dimension error from 15.29% in the previous method to 6.18% using the proposed method. Additionally, in navigation tests with a target speed of 1 m/s, the robot was able to achieve an average actual speed of 1 m/s with more efficient travel time compared to the previous control. The integration of the Kivy GUI successfully facilitated the control of multi-mode operations (manual, mapping, navigation) in real time.

Keywords: GUI Kivy, PWM Ramp Rate, ROS, Waiter-Bot.

DAFTAR ISI

Halaman

PENGAJUAN SKRIPSI ..... **iii**
ABSTRAK ..... **iv**
ABSTRACT ..... **v**
DAFTAR ISI ..... **vi**
DAFTAR GAMBAR ..... **viii**
DAFTAR TABEL ..... **ix**
DAFTAR NOTASI ..... **x**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan Penelitian 2
1.4 Manfaat Penelitian 2
1.5 Tinjauan Pustaka 2
1.5.1 Penelitian Terkait 2
1.5.2 Robot Pelayan (Service Robot) 3
1.5.3 Kinematik Robot Beroda (Differential Drive) 4
1.5.4 Robot Operating System (ROS) 5
1.5.4.1 Node dan ROS Master 6
1.5.4.2 Komunikasi Topics dan Messages 6
1.5.4.3 Publisher dan Subscriber 6
1.5.5 Sistem Navigasi dan Pemetaan 7
1.5.5.1 Sensor Fusion 7
1.5.5.2 Visual Odometry (Kinect dan RTAB-Map) 7
1.5.5.3 SLAM (Simultaneous Localization and Mapping) 7
1.5.5.4 Sistem Navigasi (AMCL dan DWA Local Planner) 7
1.5.6 Sistem Kendali Motor dan PWM Ramp Rate 8
1.5.6.1 Pulse Width Modulation (PWM) 8
1.5.6.2 PWM Ramp Rate 9
1.5.7 Interaksi Manusia dan Komputer (Human Machine Interface) 9
1.5.7.1 Kivy Framework 10
1.5.7.2 Front-End dan Back-End pada Sistem Robot 10
BAB II METODE PENELITIAN **11**
2.1 Perancangan Umum 11
2.2 Tahapan Penelitian 11

2.3 Perancangan Pada Perangkat Lunak 12
2.3.1 Perancangan di High-Level 12
2.3.1.1 Mode Controller Manual (Joystick) 14
2.3.1.2 Mapping Mode 14
2.3.1.3 Navigation Mode 14
2.3.2 Perancangan di Low-Level 14
2.4 Perancangan Perangkat Keras 15
2.4.1 Desain Mekanik Robot 15
2.4.2 Desain Elektronik 17
2.4.3 Perancangan Graphical User Interface (GUI) 18
2.4.4 Skenario Pengujian 20
2.4.4.1 Uji Kecepatan Linear dan Angular 20
2.4.4.2 Uji Mapping 20
2.4.4.3 Uji Navigasi 21
BAB III HASIL DAN PEMBAHASAN 23
3.1 Uji Performa Laju Sistem Waiter-Bot 23
3.1.1 Uji Kecepatan Linear dan Angular 23
3.1.2 Uji Mapping 26
3.1.2.1 Hasil Mapping Dengan Setting Penelitian Sebelumnya 26
3.1.2.2 Hasil Mapping Dengan Setting Kontrol Motor PWM Ramp Rate 28
3.1.3 Uji Navigasi 30
3.1.3.1 Navigasi Point A 30
3.1.3.2 Navigasi Point B 32
3.1.3.3 Navigasi Point C 34
3.2 Integrasi Graphical User Interface (GUI) dengan K 35
BAB IV PENUTUP 39
4.1 Kesimpulan 39
4.2 Saran 39
DAFTAR PUSTAKA 41
LAMPIRAN 42

DAFTAR GAMBAR

Halaman

Gambar 1 Service Robot 3
Gambar 2 Differential Drive..... 5
Gambar 3 Robot Operating System..... 5
Gambar 4 Sistem Kendali 8
Gambar 5 Pulse Width Modulation (PWM) 8
Gambar 6 PWM Ramp Rate 9
Gambar 7 Human Machine Interface 9
Gambar 8 Diagram Alir Penelitian..... 11
Gambar 9 Perancangan di High-Level Gerak Motor 13
Gambar 10 Perancangan di Low-Level Gerak Motor..... 15
Gambar 11 Desain Waiter-Bot 16
Gambar 12 Diagram Interkoneksi 17
Gambar 13 Diagram Skematik..... 18
Gambar 14 Arsitektur Hubungan GUI ke robot 19
Gambar 15 Perancangan GUI Waiter-Bot..... 19
Gambar 16 Visualisasi Peta Aktual 20
Gambar 17 Pengukuran Peta di RViz 21
Gambar 18 Jalur Navigasi Point A 21
Gambar 19 Jalur Navigasi Point B 22
Gambar 20 Jalur Navigasi Point C..... 22
Gambar 21 Grafik Kecepatan Linear 0.3 m/s..... 23
Gambar 22 Grafik Kecepatan Linear 0.5 m/s..... 23
Gambar 23 Grafik Kecepatan Linear 1 m/s..... 24
Gambar 24 Grafik Kecepatan Linear 1.5 m/s..... 24
Gambar 25 Grafik Kecepatan Angular 0.5 m/s 25
Gambar 26 Grafik Kecepatan Angular 1 m/s 25
Gambar 27 Denah Ruangan Pengujian..... 26
Gambar 28 Hasil Mapping Kontrol Motor Penelitian Sebelumnya 26
Gambar 29 Perbandingan Mapping Setting Kontrol Motor Penelitian sebelumnya dan Ground Truth..... 27
Gambar 30 Hasil Mapping Kontrol Motor PWM Ramp Rate 28
Gambar 31 Perbandingan Mapping Setting Kontrol Motor PWM Ramp Rate dan Ground Truht..... 29
Gambar 32 Jarak Navigasi Point A 31
Gambar 33 Grafik Navigasi Point A Kecepatan 0.22 m/s..... 31
Gambar 34 Grafik Navigasi Point A Kecepatan 0.5 m/s..... 31
Gambar 35 Grafik Navigasi Point A Kecepatan 1 m/s..... 32
Gambar 36 Jarak Navigasi Point B 32
Gambar 37 Grafik Navigasi Point B Kecepatan 0.22 m/s 33
Gambar 38 Grafik Navigasi Point B Kecepatan 0.5 m/s 33
Gambar 39 Grafik Navigasi Point B Kecepatan 1 m/s 33
Gambar 40 Jarak Navigasi Point C..... 34
Gambar 41 Grafik Navigasi Point C Kecepatan 0.22 m/s 34
Gambar 42 Grafik Navigasi Point C Kecepatan 0.5 m/s 35
Gambar 43 Grafik Navigasi Point C Kecepatan 1 m/s 35
Gambar 44 Graphical User Interface Waiter-Bot ..... **Error! Bookmark not defined.**

DAFTAR TABEL

Halaman

Tabel 1 Hasil Mapping Setting Kontrol Motor Konvensional dan Ground Truth.....27
Tabel 2 Hasil Mapping Setting PWM Ramp Rate dan Ground Truth.....29

DAFTAR NOTASI

BAB I

PENDAHULUAN

1.1 Latar Belakang

Waiter Robot atau robot pelayan adalah salah satu jenis robot yang telah diperkenalkan pada awal tahun 2000 di berbagai industry makanan dan minuman di beberapa negara (Quigley et al., n.d.). Robot pelayan atau waiter-bot dirancang membantu distribusi logistik, makanan di restoran cepat saji dan lingkungan rumah sakit (Prayoga & Kurniawan, 2018). Menurut Bruno Siciliano (2016), tantangan utama dalam merealisasikan robot layanan yang handal Adalah 1) kemampuan robot untuk bernavigasi secara otonom pada lingkungan dinamis 2) Kemampuan berinteraksi yang intuitif dengan pengguna (human-robot interaction).

Dalam menghadapi tantangan navigasi dan interaksi, diperlukan robot yang memiliki pergerakan yang baik dan halus, selain itu antarmuka pengguna atau Graphical User Interface (GUI) memegang peranan sebagai jembatan antara operator dan sistem robot (Yoga Prihastomo & Winanti, 2024). Pengembangan antarmuka yang responsif sering kali menghadapi tantangan integrasi platform. Singla (2023) dan Lundh (2022) mengusulkan penggunaan kerangka kerja python seperti Kivy untuk membangun aplikasi kontrol yang fleksibel dan dapat berjalan secara multi-platform, memungkinkan operator untuk memantau status robot dan mengganti mode operasi secara real-time.

Pada penelitian sebelumnya, Salam (2024) mengembangkan sistem navigasi robot otonom waiter-bot berbasis Robot Operating System (ROS) untuk pengantaran logistic secara autonomous dalam mencapai misi dan penelitian lanjutan Faizal (2025) mengembangkan sistem pemetaan Waiter-Bot menggunakan metode sensor fusion diantaranya Rotary Encoders, Inertial Measurement Unit (IMU) dan Kinect agar menghasilkan peta yang optimal digunakan. Waiter-Bot mampu melakukan pemetaan dan pergerakan secara otonom, namun terdapat beberapa kendala yang menjadi isu kritikal pada waiter-bot yaitu kendala mekanis pada sistem kendali gerak yang ditemukan tercatat kasar akibat sinyal kontrol motor yang mendadak. Borenstein & Feng (1996) dalam analisis mendalam mengenai sistem odometry errors menjelaskan bahwa percepatan atau perlambatan mendadak pada robot beroda adalah penyebab utama terjadinya selip roda. Selip ini sangat fatal bagi odometri karena menyebabkan ketidaksesuaian antara data putaran roda yang dibaca encoder dengan pergerakan aktual robot di lantai. Akumulasi kesalahan odometri ini dapat menyebabkan robot gagal mencapai titik target dengan presisi. Dari sisi mapping, pembacaan odometeri buruk menyebabkan robot sering kehilangan orientasi (lost localization) saat berada di tengah ruangan besar, kualitas peta grid (occupancy grid map) sangat rentan terhadapt drift odometri dimana kesalahan kecil pada estimasi sudut (angular error) dapat mengakibatkan peta yang dihasilkan terdistorsi (Siegwart et al., 2011).

Untuk mengatasi permasalahan gerak motor yang kasar diperlukan teknik kendali motor yang efisien dan untuk meningkatkan pengalaman interaksi robot dengan pengguna diperlukan GUI sebagai perangkat antarmuka pengguna.

Berdasarkan uraian di atas, Penelitian ini bertujuan untuk mengimpelementasikan PWM ramp rate untuk kontrol gerak motor yang lebih halus guna meningkatkan akurasi pemetaan dan navigasi otonom, serta mengintegrasikan sistem tersebut ke dalam antarmuka pengguna (GUI) Kivy untuk pengoperasian yang lebih baik.

1.2 Rumusan Masalah

1. Bagaimana laju performa sistem Waiter-Bot dengan implementasi PWM ramp rate?
2. Bagaimana implementasi Kivy pada sistem GUI pada Waiter-Bot?

1.3 Tujuan Penelitian

1. Mengimplementasikan pendekatan PWM ramp rate dengan sistem kendali
2. Menganalisis performa pemetaan setelah pengintegrasian PWM ramp rate
3. Menguji performa sistem Waiter-Bot secara keseluruhan
4. Mengimplementasikan aplikasi antarmuka (GUI) pada Waiter-Bot.

1.4 Manfaat Penelitian

1. Meningkatkan kehalusan pergerakan Waiter-Bot
2. Memudahkan Penggunaan Waiter-Bot

1.5 Tinjauan Pustaka

1.5.1 Penelitian Terkait

RANCANG BANGUN SISTEM PEMETAAN DAN LOKALISASI BERBASIS ALGORITMA SLAM MENGGUNAKAN DEPTH SENSOR KINECT PADA MOBILE ROBOT – Fariz Achmad Faizal (2025)

Memfokuskan penelitiannya pada rancangan sistem pemetaan dan lokalisasi pada mobile robot menggunakan algoritma SLAM (Simultaneous Localization and Mapping). Sistem ini mengintegrasikan sensor visual depth camera (Kinect Xbox 360) dengan metode sensor fusion (menggabungkan data encoder dan IMU). Penelitian ini berhasil mendemonstrasikan kemampuan robot dalam memetakan ruangan skala kecil dan skala besar.

RANCANG BANGUN SISTEM NAVIGASI ROBOT OTONOM WAITER BOT BERBASIS ROBOT OPERATING SYSTEM – Abd. Salam (2024)

Melakukan rancang bangun sistem navigasi otonom pada robot waiter-bot berbasis ROS dengan Navigation Stack dan Dynamic Window Approach (DWA). Robot dilengkapi dengan kemampuan mengenali batas jalur lintasan dan melakukan navigasi menuju goal yang telah ditentukan dalam peta digital

SISTEM NAVIGASI PADA PROTOTIPE ROBOT KURSI BERODA UNTUK PENYANDANG DISABILITAS - Muhammad Takbir Machmud (2019)

Mengembangkan sebuah prototipe robot kursi roda yang dilengkapi sistem navigasi untuk penyandang disabilitas. Penelitian ini memadukan mekanisme pandu arah berbasis gambar menggunakan sensor kinect dan pandu arah berbasis suara. Fokus penelitian ini pada aspek bantuan disabilitas (assistive device).

INTRODUCTION TO AUTONOMOUS MOBILE ROBOTS SECOND EDITION - Roland Siegwart, Illah R. Nourbakhsh, and Davide Scaramuzza (2011)

Dalam literatur standar robotika, menjelaskan secara teoritis bahwa perubahan kecepatan yang mendadak atau jerk (sentakan) dapat menghasilkan gaya inersia yang besar pada bodi robot. Gaya ini memicu terjadinya osilasi atau guncangan hebat pada struktur robot, yang tidak hanya merusak kenyamanan tetapi juga menyebabkan sensor

(seperti kamera atau LiDAR) kehilangan kestabilan pembacaan data, sehingga mempersulit lokalisasi robot.

1.5.2 Robot Pelayan (Service Robot)

Robotika adalah cabang ilmu interdisipliner yang menggabungkan teknik mesin, teknik listrik, dan ilmu komputer yang berhubungan dengan desain, konstruksi, operasi, dan aplikasi robot. Dalam perkembangannya, definisi robot telah bergeser dari sekadar mesin otomatis yang diprogram ulang (reprogrammable) menjadi sistem cerdas yang mampu mempersepsi lingkungan dan mengambil keputusan secara otonom. Struktur dasar robot dimodifikasi untuk mendukung mobilitas di lingkungan indoor maupun outdoor. Menurut Anshar (2025) ada lima komponen yang membangun robot: perwujudan, persepsi, aksi, otak dan kekuatan, dan otonomi.

Klasifikasi robot secara global dibagi menjadi dua domain utama: robot industri dan robot layanan. robot pelayan (Service Robot) didefinisikan sebagai robot yang melakukan tugas-tugas yang bermanfaat bagi manusia atau peralatan, tidak termasuk aplikasi otomasi industri manufaktur. dalam studi tinjauan komprehensifnya mengkategorikan robot pelayan menjadi dua jenis:

1. Robot Pelayan Personal (Personal Service Robot), digunakan untuk kepentingan individu non-komersial. Contohnya robot pembersih lantai (vacuum cleaner) dan robot pendamping edukasi.
2. 2. Robot Pelayan Profesional (Professional Service Robot), dioperasikan untuk tujuan komersial dan sering dikelola oleh operator terlatih. Contohnya robot logistik gudang, robot medis, dan lainnya (Gonzalez-aguirre et al., 2021).

Waiter-Bot termasuk dalam robot kategori pelayan professional, tantangan utama bagi robot jenis ini adalah mobilitas. Robot harus mampu berpindah dari titik A ke titik B (navigasi) tanpa menabrak objek statis maupun dinamis, serta memiliki tingkat otonomi yang cukup untuk menangani ketidakpastian lingkungan tanpa intervensi manusia yang terus-menerus.

Gambar 1 Service Robot

1.5.3 Kinematik Robot Beroda (Differential Drive)

Mekanisme pergerakan (locomotion) adalah aspek paling fundamental dalam desain robot mobil (mobile robot). Untuk aplikasi dalam ruangan (indoor) yang memiliki permukaan datar, differential drive adalah pilihan yang paling efisien dari segi energi dan pengendalian. mekanisme ini terdiri dari dua roda penggerak independen yang terletak pada satu sumbu putar yang sama, dilengkapi dengan satu atau lebih roda kastor (caster wheel) pasif sebagai penyeimbang stabilitas statis (Lynch & Park, 2017). Keunggulan utama mekanisme ini adalah sifatnya yang meskipun robot tidak dapat bergerak secara lateral (menyamping) secara instan, ia memiliki kemampuan yaitu dapat berputar 360 derajat di tempat dengan memutar kedua roda pada kecepatan yang sama tetapi berlawanan arah. Kemampuan manuver ini sangat krusial untuk operasional di lingkungan yang sempit dan penuh rintangan.

$$VR = r \cdot \theta R \qquad 1$$

$$VL = r \cdot \theta L \qquad 2$$

r : Jari-jari roda penggerak (meter)

L : Jarak antara pusat roda kiri dan roda kanan (meter)

θR : Kecepatan sudut roda kanan (rad/s)

θL : Kecepatan sudut roda kiri (rad/s)

Dimana,

$$VL = \frac{Vr + Vl}{Vr - vl} = \frac{r}{2}\,(\theta R + \theta L) \qquad 3$$

vr : Kecepatan roda kanan terhadap lantai

vl : Kecepatan roda kiri terhadap lantai

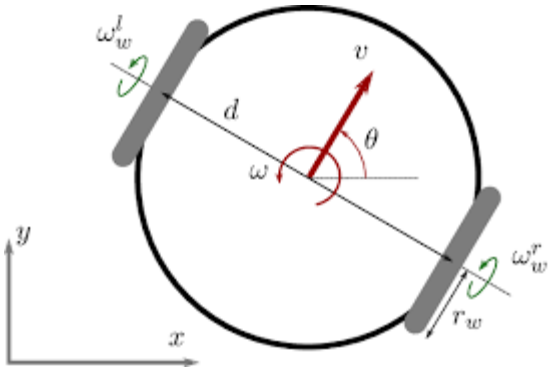
Kecepatan rotasi/angular robot (ω) disekitar pusat sumbu sesaat (Instantaneous Center of Curvature) dihitung dari selisih kecepatan roda dibagi dengan lebar wheelbase:

$$\omega = \frac{Vr - Vl}{L} = \frac{r}{L}\,(\theta R - \theta L) \qquad 4$$

Kinematika balik diperlukan dalam sistem kendali untuk mengonversi perintah kecepatan target robot (v target,ωtarget) yang dihasilkan oleh algoritma navigasi ROS menjadi kecepatan putar motor yang harus dieksekusi.

$$Vr = v\,target + \frac{\omega\,target \cdot L}{2} \qquad 5$$

$$Vl = v\,target - \frac{\omega\,target \cdot L}{2} \qquad 6$$



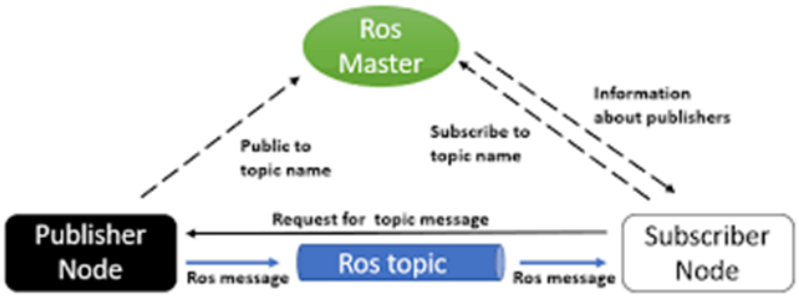
Gambar 2 Differential Drive

1.5.4 Robot Operating System (ROS)

ROS adalah Sistem Operasi Robot bersifat open source yang telah memiliki beberapa library untuk membuat software robot. ROS bertujuan untuk memudahkan pengembang robot tanpa harus membuat kode dari awal serta dapat dikembangkan bersama-sama (Jalil, 2018).

Dalam pengembangan sistem robotika modern yang kompleks, integrasi antara berbagai perangkat keras (sensor, aktuator) dan perangkat lunak (algoritma persepsi, navigasi) merupakan tantangan utama. Untuk mengatasi hal ini, penelitian ini memanfaatkan Robot Operating System (ROS). ROS bukanlah sistem operasi dalam pengertian tradisional (seperti Windows atau Linux), melainkan sebuah meta-operating system atau perangkat lunak penengah (middleware) yang berjalan di atas sistem operasi host (biasanya Ubuntu Linux). ROS menyediakan layanan standar yang diharapkan dari sebuah sistem operasi, termasuk abstraksi perangkat keras, pengendalian perangkat tingkat rendah (low-level device control), fungsi umum ROS biasanya digunakan sebagai penyampaian pesan antar-proses (inter-process communication), dan manajemen package.

Keunggulan utama ROS adalah arsitekturnya yang bersifat modular dan terdistribusi (peer-to-peer). Sistem ROS terdiri dari sejumlah proses independen yang disebut Nodes, yang saling berkomunikasi melalui kerangka kerja komputasi graf (Computation Graph) (Joseph, 2015) . Arsitektur ini memungkinkan pengembangan sistem yang robust, di mana kegagalan pada satu node (misalnya node kamera mati) tidak serta merta mematikan seluruh sistem robot.



Gambar 3 Robot Operating System

1.5.4.1 Node dan ROS Master

Elemen komputasi dasar dalam ROS adalah Node. Node adalah sebuah proses eksekusi (executable) yang melakukan komputasi tertentu. Dalam konteks Waiter-Bot pada penelitian ini, sistem terdiri dari puluhan node yang berjalan secara paralel, contohnya: .

1. Node untuk mengelola driver sensor Kinect (freenect_node).
2. Node untuk algoritma pemetaan (slam_gmapping).
3. Node untuk antarmuka mikrokontroler (rosserial_python).
4. Node untuk logika navigasi (move_base).

Agar node-node tersebut dapat saling menemukan dan berkomunikasi, diperlukan sebuah entitas pusat yang disebut ROS Master (dijalankan dengan perintah roscore). ROS Master berfungsi sebagai Name Service. Ketika sebuah node baru aktif, ia akan mendaftarkan nama dan jenis informasi yang dimilikinya ke ROS Master. Tanpa ROS Master yang berjalan, node tidak akan bisa saling mendeteksi, dan sistem robot tidak dapat beroperasi.

1.5.4.2 Komunikasi Topics dan Messages

Komunikasi data antar-node dalam ROS dilakukan melalui mekanisme streaming data yang disebut Topics. Topic adalah sebuah nama unik (bus) yang digunakan node untuk bertukar pesan.

Message atau pesan berupa data yang dikirimkan melalui Topic dibungkus dalam struktur data yang disebut Message (.msg). Pesan ini dapat berupa tipe data primitif (integer, float, boolean) atau struktur data yang kompleks. Contoh pesan yang krusial dalam penelitian ini adalah geometry_msgs/Twist digunakan untuk mengirim perintah kecepatan linear dan angular ke motor. sensor_msgs/LaserScan yang digunakan untuk mengirim data jarak dari sensor Kinect ke algoritma SLAM. Serta nav_msgs/Odometry digunakan untuk mengirim estimasi posisi robot dari encoder

Topic mengimplementasikan komunikasi many-to-many. Artinya, satu Topic dapat memiliki banyak pengirim dan banyak penerima. Sifat komunikasi Topic adalah unidirectional (satu arah) dan berkelanjutan (streaming), yang sangat cocok untuk data sensor yang memiliki frekuensi tinggi, seperti data kamera atau encoder

1.5.4.3 Publisher dan Subscriber

Mekanisme pertukaran data pada ROS didasarkan pada pola desain Publisher/Subscriber. Pola ini memisahkan pengirim data (Publisher) dari penerima data (Subscriber) demi efisiensi dan fleksibilitas sistem.

Publisher adalah node yang menghasilkan data dan mengirimkannya ke Topic tertentu. Sebagai contoh, node rosserial pada robot ini bertindak sebagai Publisher yang menerbitkan data odometri (/odom) secara terus-menerus. Publisher tidak perlu mengetahui siapa yang akan membaca datanya, ia hanya bertugas menyiarkan data ke Topic

Subscriber adalah node yang membutuhkan data dan "subscribe" ke Topic tertentu. Sebagai contoh, node gmapping bertindak sebagai Subscriber yang mendengarkan data dari Topic /scan (data laser) dan /odom (data posisi) untuk menyusun peta. Ketika data baru tersedia di Topic tersebut, fungsi callback pada Subscriber akan dieksekusi untuk memproses data. Secara teknis, proses komunikasi ROS sebagai berikut:

1. Publisher memberi tahu ROS Master bahwa ia akan menerbitkan data ke Topic.
2. Subscriber memberi tahu ROS Master bahwa ia ingin membaca data dari Topic.
3. ROS Master mencocokkan nama Topic tersebut dan memberikan alamat dan Port milik Publisher kepada Subscriber.
4. Setelah mendapatkan alamat, Subscriber menghubungi Publisher secara langsung.
5. Data kemudian dikirimkan langsung dari Publisher ke Subscriber

Mekanisme ini sangat efisien untuk mentransmisikan data bandwidth tinggi seperti video atau point cloud pada robot, serta meminimalkan latensi (delay) yang sangat kritikal dalam sistem kendali real-time (Kane & Kane, 2018)

1.5.5 Sistem Navigasi dan Pemetaan

Kemampuan navigasi otonom pada waiter-bot bergantung pada integrasi kompleks antara persepsi sensor, estimasi posisi, dan perencanaan jalur. Dalam arsitektur Robot Operating System (ROS), sistem ini dibagi menjadi beberapa subsistem utama yaitu Sensor Fusion, Visual Odometry, SLAM, dan Path Planning.

1.5.5.1 Sensor Fusion

Untuk mendapatkan estimasi posisi robot yang akurat, tidak cukup hanya mengandalkan satu jenis sensor. Metode Sensor Fusion digunakan untuk menggabungkan data dari berbagai sumber guna meminimalkan ketidakpastian (uncertainty). Fusi dilakukan menggunakan algoritma Extended Kalman Filter (EKF) melalui paket robot_pose_ekf di ROS. Rotary Encoder memberikan data odometri interoseptif berdasarkan putaran roda. Keunggulannya adalah frekuensi update yang tinggi dan presisi dalam jangka pendek. Inertial Measurement Unit (IMU) mengukur percepatan linear dan kecepatan sudut (angular velocity) menggunakan akselerometer dan giroskop. IMU sangat efektif dalam mendeteksi rotasi robot yang cepat yang mungkin tidak terdeteksi oleh encoder saat roda tergelincir. Namun, IMU memiliki kelemahan berupa bias drift yang menyebabkan kesalahan posisi membesar seiring waktu jika tidak dikoreksi.

1.5.5.2 Visual Odometry (Kinect dan RTAB-Map)

Selain odometri roda dan inersia, penelitian ini menerapkan Visual Odometry (VO) sebagai sumber estimasi posisi tambahan, khususnya untuk mengatasi situasi di mana roda mengalami selip total. VO bekerja dengan menganalisis urutan gambar dari kamera untuk mengestimasi pergerakan robot.

1.5.5.3 SLAM (Simultaneous Localization and Mapping)

SLAM adalah metode komputasi yang memungkinkan robot membangun peta lingkungan yang belum diketahui sekaligus memperkirakan posisinya di dalam peta tersebut secara bersamaan, robot butuh peta untuk mengetahui posisi, tetapi butuh posisi yang akurat untuk menggambar peta.

1.5.5.4 Sistem Navigasi (AMCL dan DWA Local Planner)

Setelah peta terbentuk, robot memerlukan sistem navigasi untuk bergerak dari titik awal ke titik tujuan (goal) secara otonom. AMCL (Adaptive Monte Carlo Localization) adalah algoritma probabilistik untuk melacak posisi robot pada peta yang sudah ada. Berbeda dengan SLAM yang membangun peta, AMCL hanya mencocokkan data sensor saat ini dengan peta statis.

Perencanaan Jalur (Path Planning) Sistem navigasi ROS (Navigation Stack) menggunakan dua lapis perencanaan yaitu Global Planner dalam menghitung jalur

terpendek dari posisi robot ke tujuan menggunakan algoritma Jalur ini bersifat statis dan mengabaikan rintangan dinamis baru. Sedangkan Local Planner Dynamic Window Approach (DWA) bertugas mengendalikan motor untuk mengikuti jalur global sambil menghindari halangan dinamis.

1.5.6 Sistem Kendali Motor dan PWM Ramp Rate

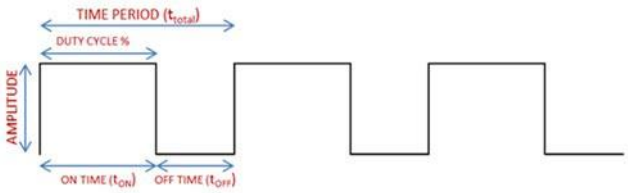
Pada robot bergerak (mobile robot), sistem kendali motor bertindak sebagai jembatan antara perintah navigasi tingkat tinggi (High-Level) yang dihasilkan oleh ROS dengan eksekusi fisik tingkat rendah (Low-Level) pada aktuator. Kualitas pergerakan robot sangat bergantung pada bagaimana sinyal kendali dikelola untuk menangani dinamika beban dan inersia.

Gambar 4 Sistem Kendali

1.5.6.1 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) atau Modulasi Lebar Pulsa adalah teknik yang digunakan untuk mengontrol daya yang disuplai ke perangkat listrik, khususnya motor DC, dengan cara memvariasikan lebar pulsa sinyal digital. Alih-alih memberikan tegangan analog yang bervariasi (yang tidak efisien dan menghasilkan panas berlebih pada transistor), PWM menswitch tegangan antara logika High (VCC) dan Low (GND) dengan frekuensi tinggi.

Parameter utama dalam PWM adalah Siklus Kerja (Duty Cycle), yang didefinisikan sebagai persentase waktu sinyal berada dalam kondisi ON terhadap total periode sinyal. Tegangan rata-rata yang diterima motor dapat dihitung dengan persamaan:



Gambar 5 Pulse Width Modulation (PWM)

Di mana:

Siklus kerja (0% - 100%).

Periode gelombang PWM (1/f).

Tegangan suplai input (misal: 12V).

Dengan mengatur nilai siklus kerja, mikrokontroler dapat mengatur kecepatan putar motor DC secara linear. Namun, penerapan perubahan secara instan (step change) dapat menimbulkan masalah mekanis yang serius.

1.5.6.2 PWM Ramp Rate

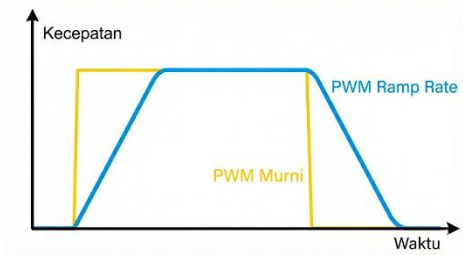
Dalam sistem kendali, Ramp Rate didefinisikan sebagai laju perubahan maksimum yang diizinkan dari suatu variabel keluaran terhadap waktu. Dalam konteks pengendalian motor robot (motor control), ramp rate secara spesifik merujuk pada batasan akselerasi (percepatan) dan deselerasi (perlambatan)

Secara sederhana, ramp rate adalah mekanisme yang mencegah motor berubah dari kecepatan 0 ke kecepatan maksimum secara instan (step change). Sebaliknya, motor dipaksa untuk menaikkan kecepatannya secara bertahap mengikuti kemiringan (slope) tertentu hingga mencapai target

Jika kita memplot grafik antara Kecepatan (v) terhadap Waktu (t), ramp rate merepresentasikan kemiringan (gradient) dari grafik tersebut. .Tanpa Ramp (Step Input) grafik berbentuk kotak tegak lurus. Perubahan kecepatan terjadi dalam waktu mendekati nol, yang secara teoritis membutuhkan gaya tak hingg a.Dengan Ramp (Trapezoidal Profile) grafik berbentuk trapesium. Sisi miring naik adalah akselerasi, bagian datar adalah kecepatan konstan, dan sisi miring turun adalah deselerasi.Secara matematis, ramp rate (R) dirumuskan sebagai:

Δv : Perubahan kecepatan (satuan unit kecepatan atau RPM).

Δt : Waktu yang dibutuhkan untuk perubahan tersebut



Gambar 6 PWM Ramp Rate

1.5.7 Interaksi Manusia dan Komputer (Human Machine Interface)

Dalam konteks robotika, antarmuka berfungsi untuk menyederhanakan kompleksitas sistem ROS yang berbasis teks (command line interface) menjadi bentuk visual yang intuitif (Graphical User Interface). Yoga Prihastomo dan Winanti (2024) menyatakan bahwa tren desain industri saat ini menuntut adanya visualisasi data real-time yang memungkinkan operator mengambil keputusan cepat tanpa harus memahami kode pemrograman yang rumit.

Gambar 7 Human Machine Interface

1.5.7.1 Kivy Framework

Untuk membangun antarmuka pada penelitian ini, digunakan kerangka kerja (framework) Kivy. Kivy adalah pustaka open-source berbasis Python yang dirancang untuk pengembangan aplikasi multitouch dan lintas platform (cross-platform). Kivy sangat efektif untuk menjembatani komunikasi antara pengguna awam dengan sistem robot yang kompleks karena fleksibilitasnya dalam menangani event loop secara asinkron

1.5.7.2 Front-End dan Back-End pada Sistem Robot

` Dalam rekayasa perangkat lunak aplikasi robot ini, diterapkan pola arsitektur yang memisahkan antara lapisan presentasi (Front-End) dan lapisan logika bisnis (Back-End). Pemisahan ini krusial untuk menjaga agar antarmuka tidak mengalami freeze (macet) saat robot sedang melakukan komputasi berat.

Front-End adalah bagian dari perangkat lunak yang berinteraksi langsung dengan pengguna. Front-End bekerja dalam sebuah Main Loop yang terus-menerus memperbarui tampilan layar (misalnya 60 frame per detik). Front-End tidak boleh melakukan pemrosesan data yang berat, tugasnya hanya menerima perintah operator dan mengirimkannya ke Back-End.

Back-End adalah "otak" yang bekerja di balik layar untuk memproses perintah dan mengendalikan robot, Mengelola siklus hidup proses ROS (roscore, nodes), memilih file peluncuran (launch file) yang sesuai

BAB II

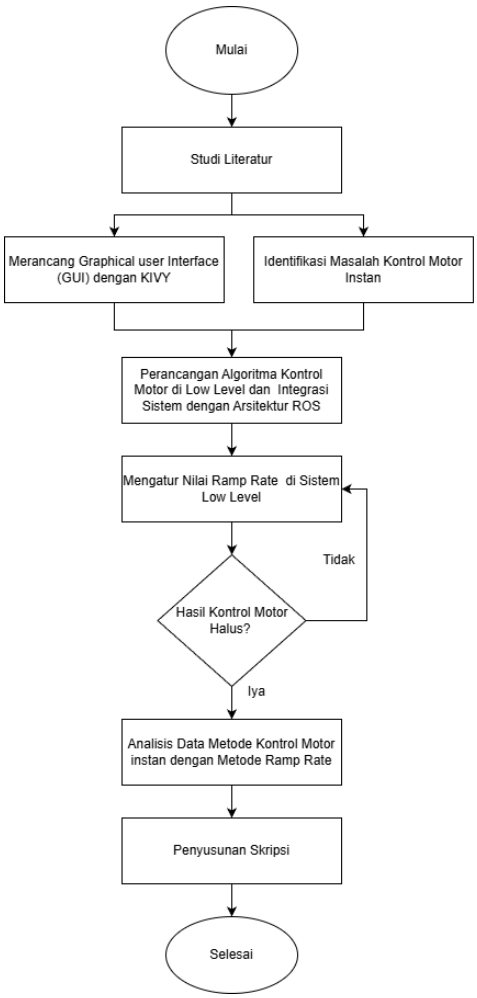
METODE PENELITIAN

2.1 Perancangan Umum

Penelitian ini dimulai pada bulan Mei 2025 hingga Januari 2026 dilaksanakan di Laboratorium Sosial Kognitif dan Robotika, Departemen Teknik Elektro, Fakultas Teknik, Universitas Hasanuddin, Gowa khususnya Gedung Centre of Technology (COT) Lantai 1. Dalam meningkatkan kualitas gerak motor digunakan metode ramp-rate dalam mengatur nilai PWM yang dikirimkan di motor untuk menghasilkan pergerakan yang lebih halus dan aman, sedangkan dari segi tampilan antarmuka digunakan framework kivy sebagai GUI dalam memudahkan pengguna sehingga tidak memerlukan pengetikan secara hardware dalam menggunakan waiter-bot.

2.2 Tahapan Penelitian

Penelitian ini dilakukan dalam beberapa tahap. Adapun diagram alir dari tahapan penelitian ini secara umum dapat dilihat pada gambar berikut:



Gambar 8 Diagram Alir Penelitian

2.3 Perancangan Pada Perangkat Lunak

Pada rancangan waiter-bot, dibagi menjadi 2 level yaitu : high-level dan low-level yang saling terhubung satu sama lain dalam mengendalikan robot khususnya gerak motor dan antarmuka pengguna

2.3.1 Perancangan di High-Level

Perancangan di High-Level Gerak Motor mencakup pengendalian Joystick, Kinect, PC (komputer), ROS. Mode controler (joystick) aktif dengan menjalankan node controller.launch, node ini terhubung dengan x360_joy_node sebagai kontrol dari joystick.

```
<node name="x360_joy_node" pkg="my_robot_pkg" type="x360_joy_node.py" output="screen"/>
```

Node ini digunakan sebagai parameter untuk menentukan kecepatan joystick dengan mengatur twist, selain itu melakukan publish data cmd_vel yang nantinya akan terbaca di low level melalui rosserial sebagai pergerakan.

```
def joy_callback(data):
    twist = Twist()
    if data.axes[7] == 1.0 :
        twist.linear.x = 0.3
    elif data.axes[7] == -1.0 :
        twist.linear.x = -0.3
    elif data.axes[6] == 1.0 :
        twist.angular.z = 0.3
    elif data.axes[6] == -1.0 :
        twist.angular.z = -0.3
    else :
       twist.linear.x = data.axes[1] * 1
       twist.angular.z = data.axes[0] * 1
    pub.publish(twist)
pub = rospy.Publisher('cmd_vel', Twist, queue_size=10)
rospy.Subscriber('joy', Joy, joy_callback)
```

Mapping mode aktif dengan menjalankan node mapping.launch, pada mode ini, mapping terdiri dari beberapa node seperti freenect.launch untuk pembacaan dari sensor kinect, RViz, dan SLAM Gmapping.

```
<include file="$(find freenect_launch)/launch/freenect.launch">
```

Dengan rviz, menu untuk pemetaan, model robot, sensor, dan node lainnya akan ditampilkan dan dengan menggunakan metode SLAM memungkinkan pemetaan secara real-time berlangsung

```
<node name="rviz" pkg="rviz" type="rviz" args="-d $(find autonomus_mobile_robot)/config/amcl_nav.rviz"/>

<node pkg="gmapping" type="slam_gmapping" name="turtlebot3_slam_gmapping" output="screen">
```

Untuk hasil dari pemetaan, disimpan di server dengan format .yaml dan .pgm

```
rosrun map_server map_saver -f my_map
```

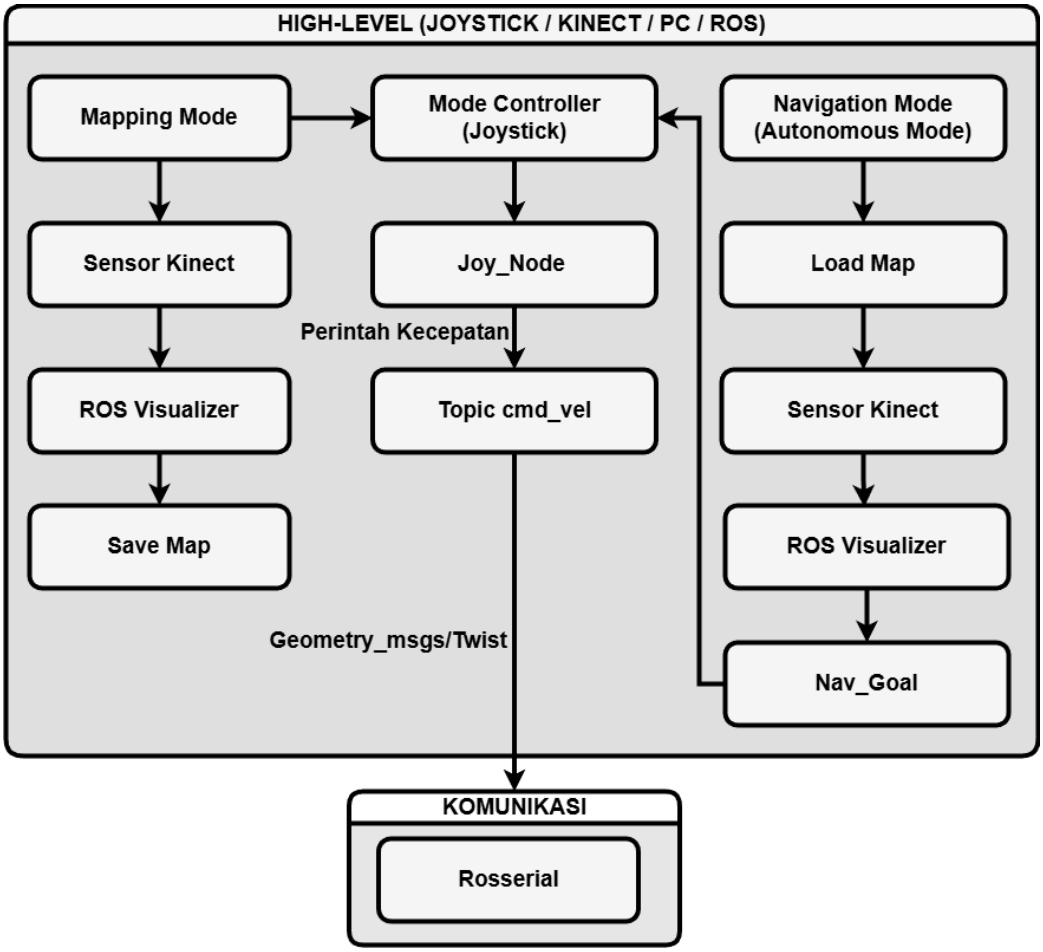
Untuk Navigation Mode (Autonomous), memiliki setting yang hampir sama dengan mode mapping, yang membedakan ialah pemilihan peta dilakukan di awal untuk navigasi

```
<node name="map_server" pkg="map_server" type="map_server" args="$(find autonomus_mobile_robot)/maps/your_map_name.yaml" />
```

Pembacaan dari sensor kinect dengan node freenect. Pada RViz ditampilkan posisi robot dan peta yang digunakan dan dengan menggunakan fitur 2d_nav_goals penentuan tujuan navigasi dari robot di aktifkan, digunakan metode navigation_stack untuk penentual global_planner dan local_planner pada node move_base.

```
<include file="$(find autonomus_mobile_robot)/launch/move_base.launch">
```

Robot mampu melakukan navigasi jika mode controller (joystick) aktif, hal ini disebabkan perintah 2d_nav_goals dari ROS hanya melakukan publish kecepatan dari cmd_vel yang tidak terhubung dengan low-level sehingga motor belum menerima perintah pergerakan. Dengan mengaktifkan mode controller (joystick), nilai cmd_vel navigasi dari ROS diterima di low-level melalui rosserial sebagai perintah motor bergerak sesuai dengan nilai cmd_vel.



Gambar 9 Perancangan di High-Level Gerak Motor

Perancangan di High-Level Gerak Motor mencakup pengendalian Joystick, Kinect, PC (komputer), ROS yang saling terhubung.

2.3.1.1 Mode Controller Manual (Joystick)

Pada modde Pada mode ini, pengendalain robot dilakukann secara manual dengan menggunakan joystick sebagai input, pergerakan analog joystick dibaca pada joy_node, node ini akan mengirimkan nilai tersebut ke topik cmd_vel. Pesan inilah yang nantinya akan terbaca sebagai gerakan dan terhubung dengan proses di low-level khususnya di mikrokontroller yang terhubung dengan komunikasi serial (rosserial).

2.3.1.2 Mapping Mode

Mapping mode memungkinkan pengguna untuk membuat peta berdasarkan ruang atau lokasi yang ditentukan, pemetaan dilakukan secara manual dengan menggerakkan robot menggunakan joystick, selain itu dibutuhkan sensor kinect untuk melakukan pendeteksian lingkungan sekitar, dan dengan menggunakan metode SLAM, pembacaan dan penyimpanan peta secara real time di tampilkan di ROS Visualizer (RVIZ), Perlu diketahui titik awal ketika mapping dimulai akan menjadi posisi awal dari robot jika menggunakan mode navigasi dan posisi ini tidak dapat dipindahkan sehingga pengguna harus tahu posisi awal dari robot. Setelah pengguna merasa peta sudah sesuai, selanjutnya menyimpan hasil peta secara keseluruhan di server.

2.3.1.3 Navigation Mode

Mode navigasi, robot melakukan pergerakan secara otomatis. Di mode ini, robot akan memuat peta yang telah dibuat di mapping mode sebelumnya, dari hasil map inipun robot harus ditempatkan di titik awal mapping. Node Controller dan Sensor Kinect memiliki peranan yang sangat penting dalam pergerakan, menentukan jalur navigasi dan pembacaan objek seperti obstacle. RVIZ akan menampilkan posisi robot dan peta yang digunakan, dan dengan tampilan ini pengguna dapat menentukan goal atau tujuan navigasi dengan menggunakan fitur 2d nav_goals.

2.3.2 Perancangan di Low-Level

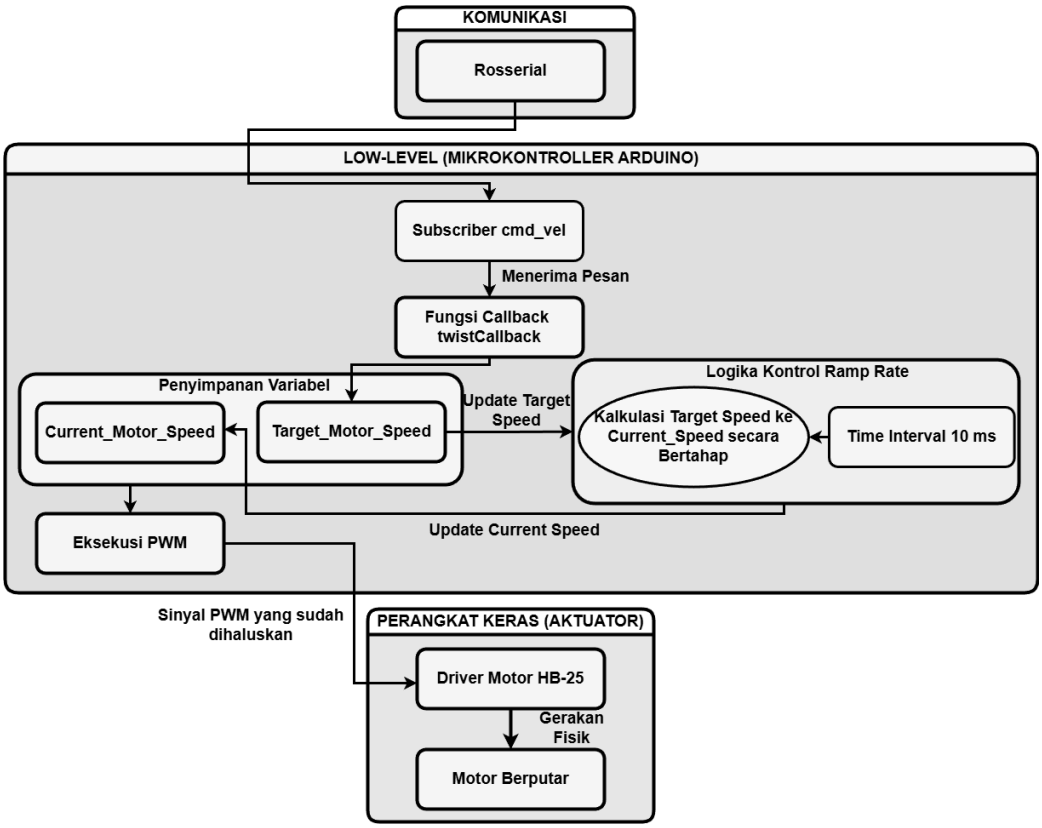
Di low-level, pesan yang dikirimkan ROS diterima dengan menggunakan komunikasi serial (rosserial), mikrokontroller (arduino) yang akan mengeksekusi perintah yang diberikan seperti perintah gerak, pembacaan sensor imu, dan encoder.

Rancangan low-level gerak motor terhubung dengan high-level (ROS) melalui komunikasi serial (rosserial). Topik cmd_vel yang ada pada ROS disubscribe di low-level sehingga mikrokontroller dapat menerima pesan seperti perintah gerak, pesan tersebut secara otomatis memicu twistCallback yang berfungsi untuk memperbarui tujuan atau target kecepatan motor (target_motor_speed) dan disimpan di variabel global selama perintah gerak diberikan. Logika kontrol ramp rate melakukan kalkulasi dalam mengatur target_motor_speed ke nilai current_motor_speed secara bertahap tiap interval 10ms. Hasil kalkulasi secara bertahap ini disimpan di global variabel (current_motor_speed) dan diubah menjadi nilai PWM, nilai inilah yang nantinya diterima driver (HB-25) dalam menggerakkan aktuator (motor).

```
ros::Subscriber<geometry_msgs::Twist> sub_twist("cmd_vel", &twistCallback);
```

Kontrol Ramp Rate diatur untuk melakukan Update kecepatan setiap 10 milidetik, dan tiap interval tersebut dilakukan kalkulasi nilai target ke nilai sekarang dengan setiap step sehingga nilai PWM yang masuk ke motor meningkat atau menurun secara perlahan mengikuti nilai target.

```
const int MOTOR_UPDATE_INTERVAL = 10;
const int SPEED_STEP = 2;
```



Gambar 10 Perancangan di Low-Level Gerak Motor

Impelemntasi algoritma penghalusan gerak pada penelitian ini mengadopsi prinsip Digital Ramp Generator dalam domain waktu diskrit, generator ini bekerja dengan mengakumulasi nilai Langkah (Δstep) untuk membentuk lintasan referensi kecepatan. Metode ini diformulakan.

$$ramp_{out}(n) = ramp_out\ (n-1) + 2, \quad jika\ ramp_out < target \quad dan\ (ramp_out\ + 2)\ <\ ramp_max \tag{7}$$

$$ramp_{out}(n) = ramp_out\ (n-1) - 2, \quad jika\ ramp_out > target \quad dan\ (ramp_out\ - 2)\ <\ ramp_min \tag{8}$$

Persamaan di atas menunjukkan bahwa keluaran (ramp_out) akan mengejar nilai target dengan kenaikan atau penurunan sebesar 2 unit (atau speed_step) pada setiap iterasi, hingga mencapai batas saturasi (ramp_min) atau (ramp_max) Metode ini secara efektif membatasi lonjakan akselerasi (dv/dt), sehingga menghasilkan profil gerak yang linear dan meminimalisir sentakan (jerk) pada aktuator robot

2.4 Perancangan Perangkat Keras

2.4.1 Desain Mekanik Robot

Struktur dasar robot dimodifikasi untuk mendukung mobilitas di lingkungan indoor maupun outdoor. Menurut Muh Anshar (2025) ada lima komponen yang membangun robot: perwujudan, persepsi, aksi, otak dan kekuatan, dan otonomi. Namun dalam hal

ini, waiter-bot masih belum otonom sepenuhnya, robot masih dioperasikan dengan teleoperated dikendalikan Sebagian oleh manusia

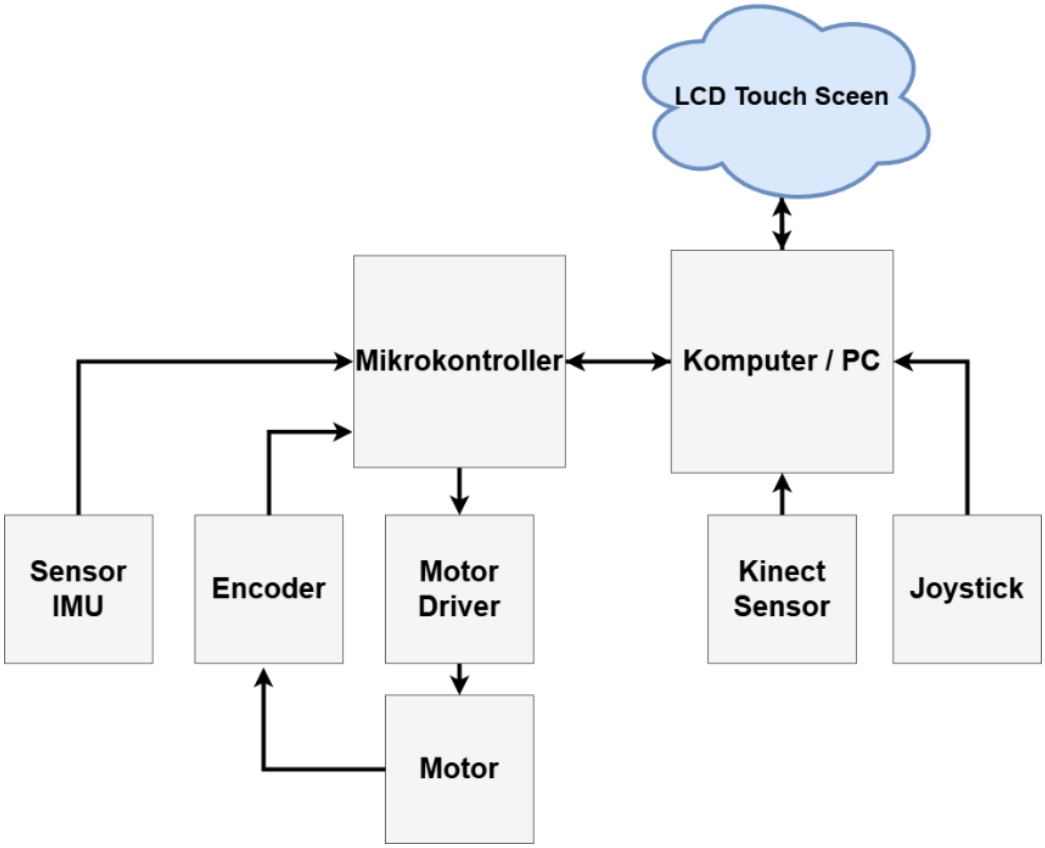
Gambar 11 Desain Waiter-Bot

Perwujudan, dapat diartikan sebagai tubuh, sebagai syarat awal menjadi robot, memungkinkan robot melakukan berbagai hal seperti berpergian suatu tempat dan menyelesaikan pekerjaannya.

Persepsi, berfungsi sebagai penginderaan robot untuk mengenali lingkungan, komponen ini terdiri dari sensor kinect dalam mendeteksi rintangan dan fitur visual untuk proses SLAM dan sensor IMU untuk memberikan data orientasi dan percepatan linear untuk mengoreksi kesalahan orientasi (angular drift) yang mungkin terjadi akibat guncangan mekanis saat robot bergerak.

Aksi, sebagai Penggerak (Aktuator dan Lokomosi) adalah mekanisme fisik yang memungkinkan robot untuk berpindah tempat dari satu lokasi ke lokasi lain dalam lingkungannya, komponen yang digunakan adalah encoder untuk memberikan umpan balik (feedback) dan dua motor DC dengan konfigurasi Differential Drive dengan driver motor. Konfigurasi ini terdiri dari dua roda utama yang terletak pada satu sumbu yang sama dan digerakkan secara independen. Mekanisme ini memungkinkan robot untuk melakukan tiga jenis gerakan dasar dengan memanipulasi kecepatan roda kiri dan roda kanan, seperti gerak maju/mundur, gerak berbelok, dan gerak putar arah ditempat.

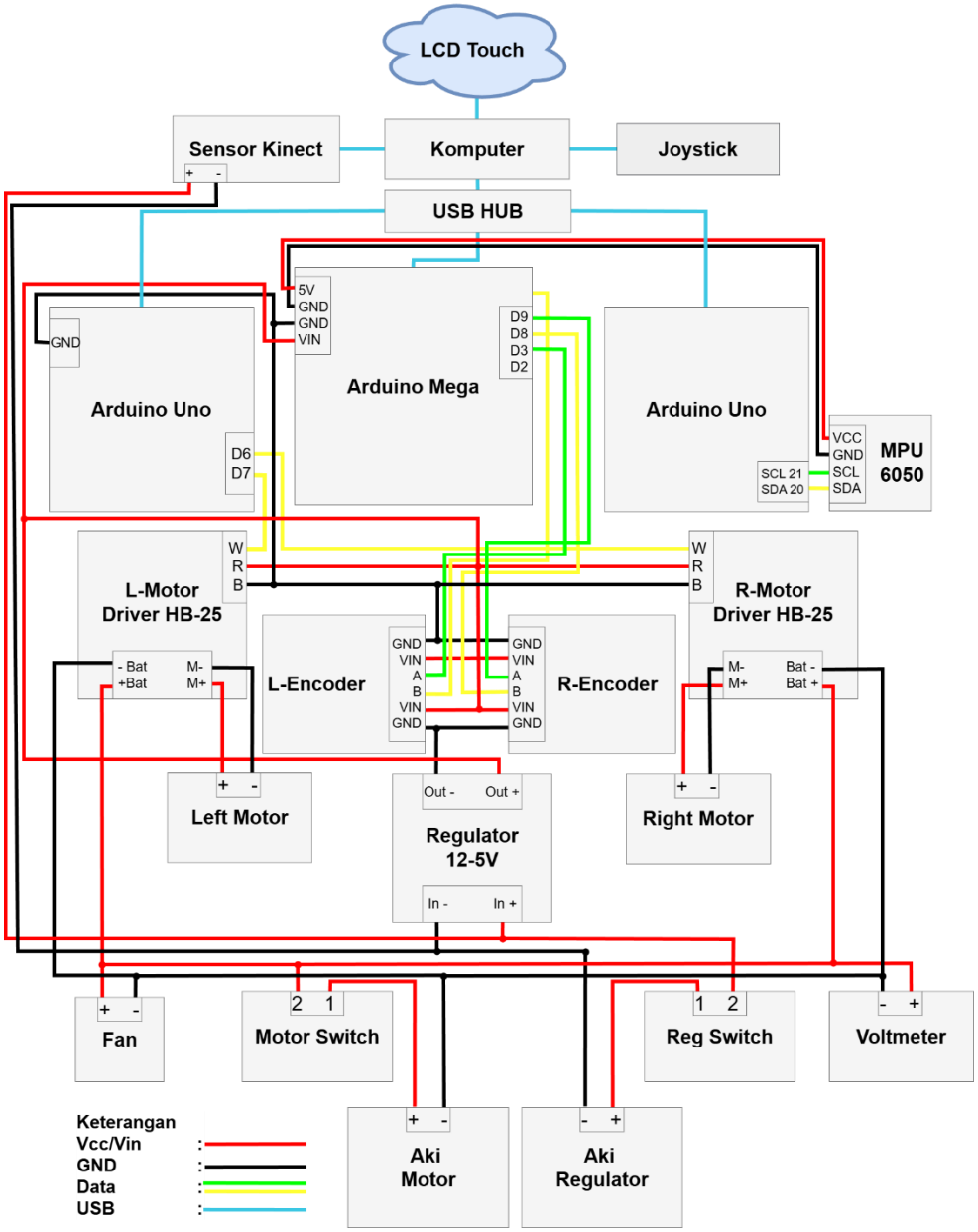
Otak dan kekuatan, robot membutuhkan otak agar dapat berfungsi dengan baik dalam hal ini komputasi, selain itu, pmbagin daya yang tepat sangat dibutuhkan agar robot bekerja dengan baik. Pada waiter-bot dibagi menjadi 2 yaitu high level controller (PC) sebagai otak utama yang menjalankan ROS, dan low level controller sebagai pengendali perangkat keras.



Gambar 12 Diagram Interkoneksi

2.4.2 Desain Elektronik

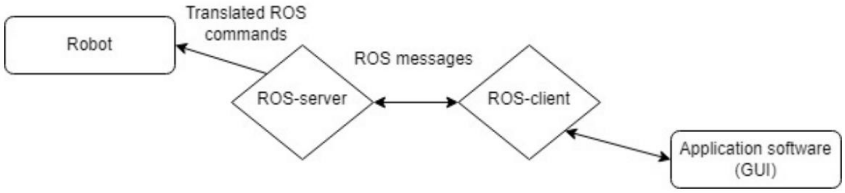
Seluruh sistem ditenagai oleh dua buah aki 12V. Satu aki digunakan untuk menyuplai daya besar ke motor DC melalui driver, sedangkan aki lainnya digunakan untuk menyuplai daya stabil ke mikrokontroller dan sensor melalui regulator tegangan. Pemisahan jalur daya ini bertujuan untuk mencegah terjadinya gangguan kinerja sensor sensitive dan mikrokontroller yang sangat penting untuk menjaga persepsi dari robot. Dilakukan penambahan interface seperti LCD menjadi bagian interaksi robot dengan operator.



Gambar 13 Diagram Skematik

2.4.3 Perancangan Graphical User Interface (GUI)

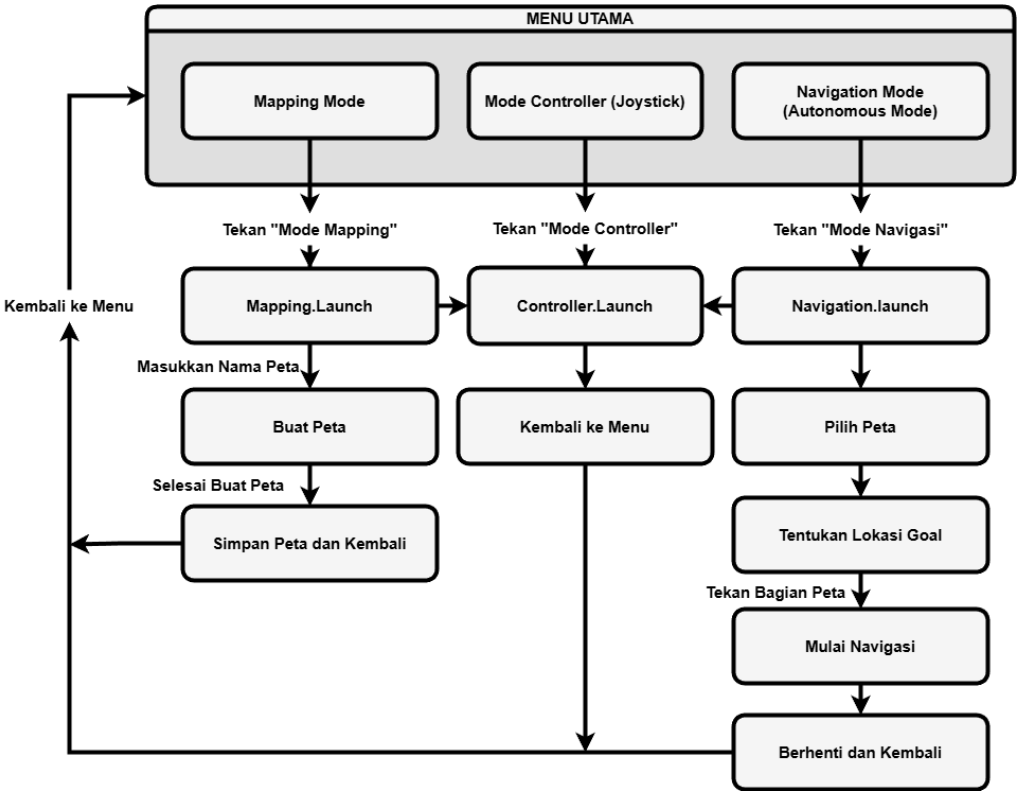
GUI merupakan tampilan antarmuka yang memungkinkan pengguna mampu berinteraksi dengan sistem komputer dengan berbagai elemen grafis seperti tombol, jendel, menu, gambar, dan lain-lain. Selain itu dengan merancang GUI, efisiensi penggunaan suatu perangkat lunak atau sistem memiliki dampak yang signifikan pada pengalaman pengguna



Gambar 14 Arsitektur Hubungan GUI ke robot

Kivy adalah sebuah framework open-source dengan menggunakan bahasa pemrograman python, menjadi salah satu solusi untuk membangun aplikasi ataupun tampilan antarmuka multi-sentuh diberbagai platform. Perangkat ini memungkinkan pengembangan yang sederhana dan fleksibel dalam membuat aplikasi yang kuat dan intuitif

Ros-server melakukan publish sebuah topik yang nantinya akan disubscribe oleh ros-client untuk menerima pesan. Di GUI, dengan menekan elemen yang diberikan perintah seperti menjalankan robot akan diterima sebagai pesan dari client dan ros-server akan menghubungkannya sebagai pergerakan di robot



Gambar 15 Perancangan GUI Waiter-Bot

Pengembangan antarmuka pengguna (GUI) pada sistem Waiter-Bot dirancang menggunakan pola arsitektur yang memisahkan antara logika tampilan (presentation

layer) dan logika pemrosesan data (application logic). Pendekatan ini bertujuan untuk meningkatkan modularitas dan responsivitas sistem. Berdasarkan kerangka kerja Kivy yang digunakan, struktur perangkat lunak dibagi menjadi dua komponen utama yaitu Front-End dan Back-End.

Sisi front-end bertanggung jawab atas visualisasi data dan interaksi langsung dengan pengguna. Menangkap event dari pengguna (seperti sentuhan pada layar atau penekanan tombol joystick virtual) dan menyajikannya dalam bentuk visual yang intuitif (Ben Shneiderman, 2016). Sisi Back-End diimplementasikan menggunakan bahasa pemrograman Python yang terintegrasi dengan pustaka rospy (ROS Python). Bagian ini berfungsi sebagai jembatan antara input pengguna dari Front-End dan sistem kendali robot. Back-End menangani pemrosesan data di balik layar.

2.4.4 Skenario Pengujian

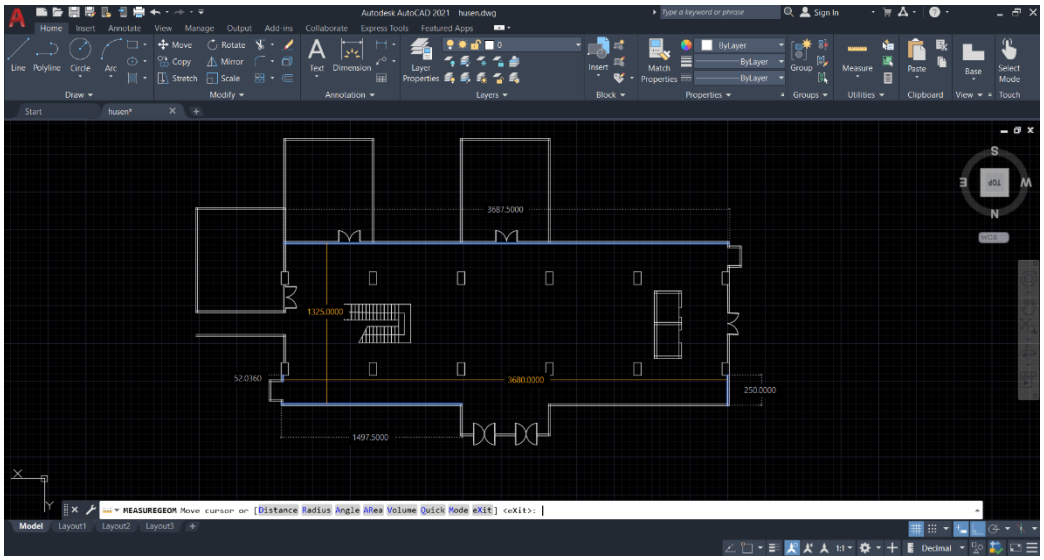
2.4.4.1 Uji Kecepatan Linear dan Angular

Uji kecepatan linear dan angular dilakukan dengan membandingkan kontrol motor konvensional yang digunakna pada penelitian sebelumnya dan kontrol motor dengan PWM Ramp Rate, Dari ROS perintah msg.linear x untuk kecepatan maju (m/s) dan msg.angular.z untuk kecepatan putar yang diinginkan (rad/s), topik dari /right_ticks dan /left_ticks dari data encoder digunakan sebagai pembacaan putaran motor untuk melihat kecepatan pergerakan motor dengan parameter kecepatan yang digunakan 0.22, 0.5, 1, 1.5 m/s yang diujikan pada lintasan sejauh 2 meter dan untuk kecepatan angular 0.5 dan 1 m/s dalam melakukan 1 putaran penuh

$$Actual\ Linear = \frac{vR + vL}{2} \qquad 9$$

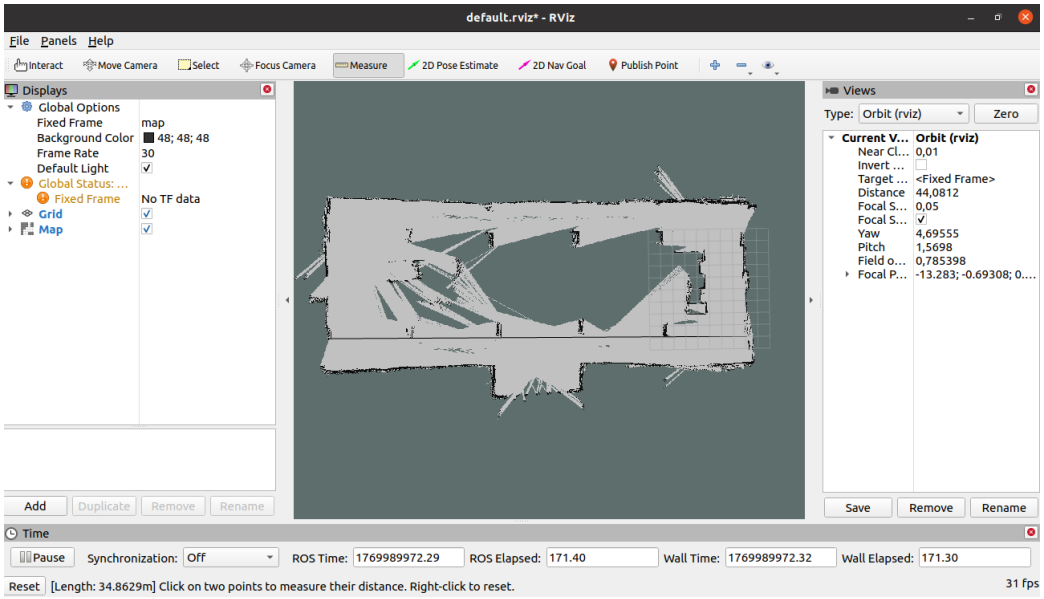
2.4.4.2 Uji Mapping

Uji mapping dilakukan dengan melakukan pengukuran ruangan pengujian secara langsung secara keseluruhan, hasil pengukuran kemudian di desainkan dengan perangkat Autodesk autocad untuk memudahkan pengukuran dan perbandingan untuk hasil mapping.



Gambar 16 Visualisasi Peta Aktual

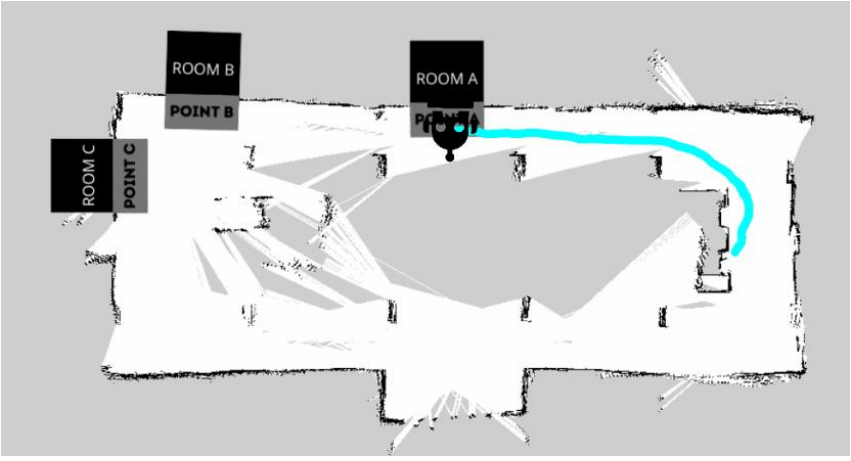
ROS telah dilengkapi dengan fitur Ros Visualizer (Rviz) sebagai dashboard yang menampilkan hasil pemetaan, selain itu terdapat fitur measure yang dapat digunakan untuk menghitung jarak dari satu titik ke titik lainnya. Fitur inilah yang digunakan untuk membandingkan hasil peta yang ada di dunia nyata dan yang dihasilkan waiter-bot.



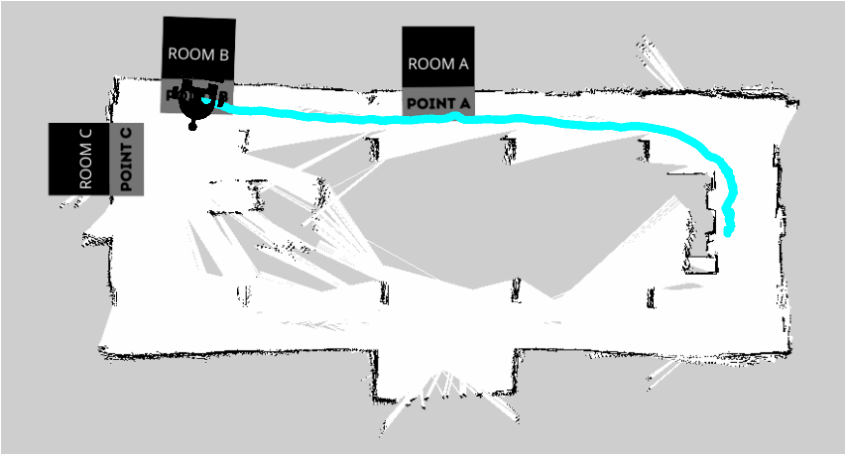
Gambar 17 Pengukuran Peta di RViz

2.4.4.3 Uji Navigasi

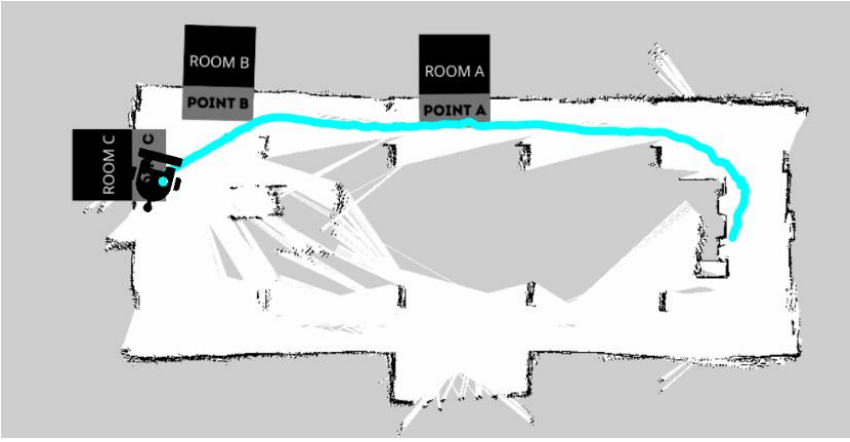
Uji navigasi dilakukan dengan menghitung pergerakan motor dengan parameter kecepatan yang digunakan 0.22, 0.5, 1 m/s. Parameter input navigasi berasal dari ROS dengan mengirimkan nilai dari cmd_vel yang berasal dari DWA Local Planner ynag berupa msg.linear x untuk kecepatan maju (m/s) dan msg.angular.z untuk kecepatan putar yang diinginkan (rad/s), topik dari /right_ticks dan /left_ticks dari data encoder untuk pembacaan pergerakan aktual motor dan topik odom untuk pembacaan koordinat estimasi Lokasi dan jalur pergerakan robot.



Gambar 18 Jalur Navigasi Point A



Gambar 19 Jalur Navigasi Point B



Gambar 20 Jalur Navigasi Point C

BAB III

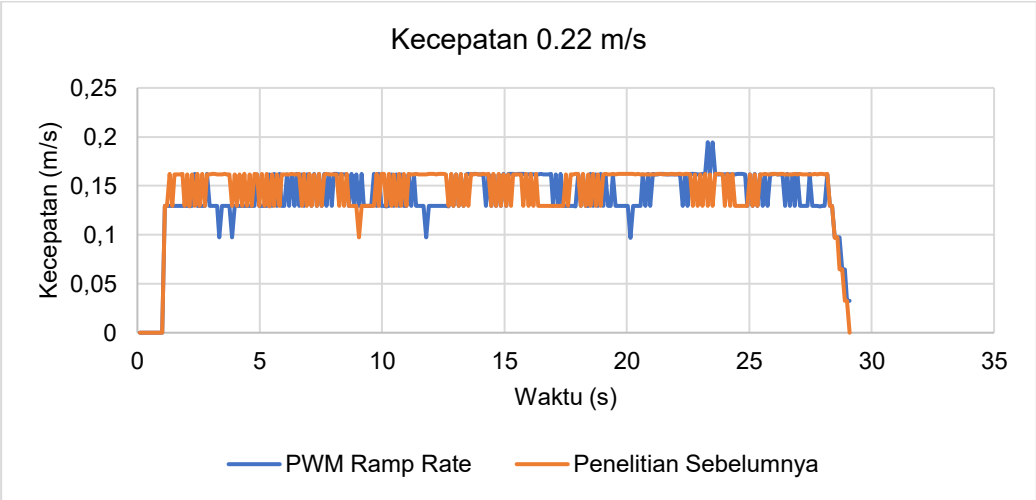
HASIL DAN PEMBAHASAN

3.1 Uji Performa Laju Sistem Waiter-Bot

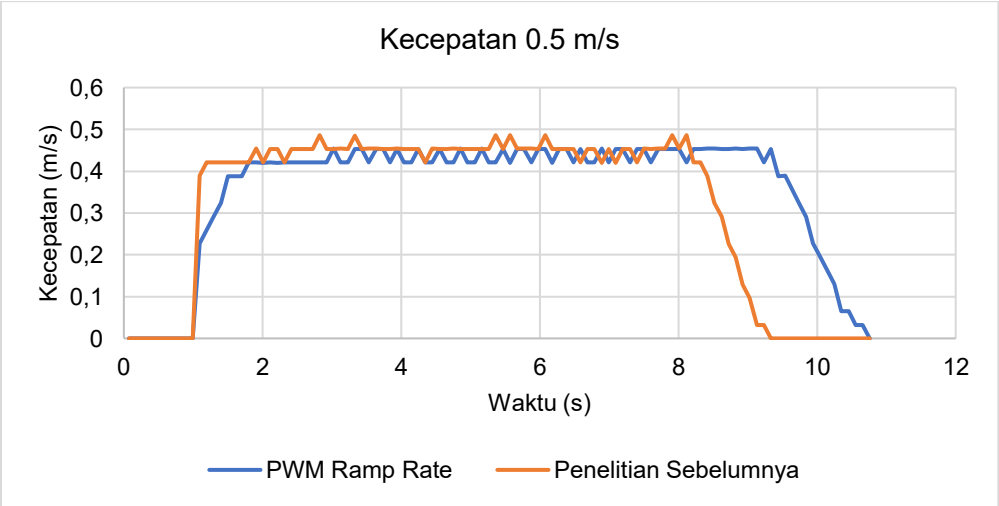
Uji performa laju sistem waiter-bot dilakukan dengan membandingkan kontrol motor instan (konvensional) yang digunakan di penelitian sebelumnya dan kontrol motor PWM Ramp Rate dengan parameter kecepatan yang berbeda.

3.1.1 Uji Kecepatan Linear dan Angular

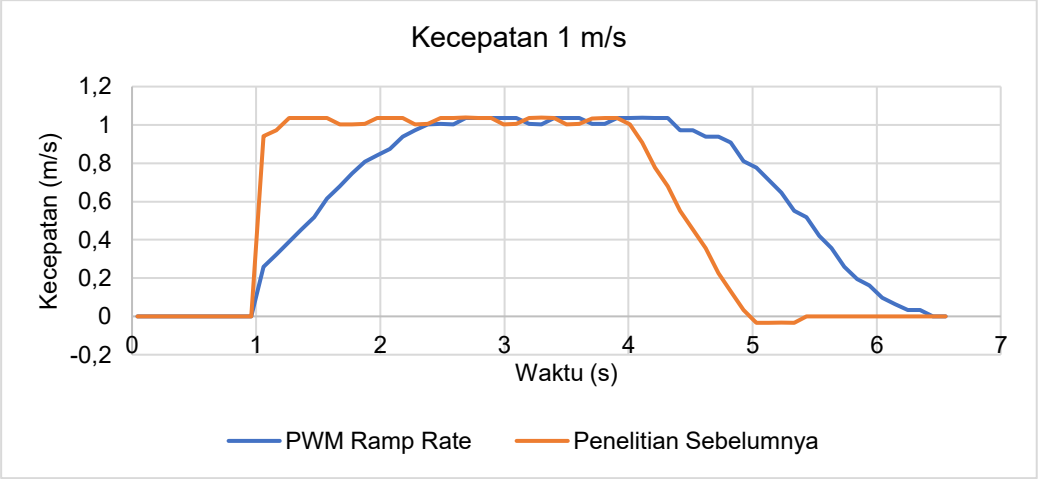
Untuk melihat grafik perbandingan pergerakan motor, dilakukan pengujian kecepatan linear dan angular dengan kontrol motor pada penelitian sebelumnya dan setelah ditingkatkan dengan PWM Ramp Rate, adapun kecepatan linear yang digunakan adalah 1.5, 1, 0.5, 0.22 m/s dengan jarak tempuh sejauh 2 meter.



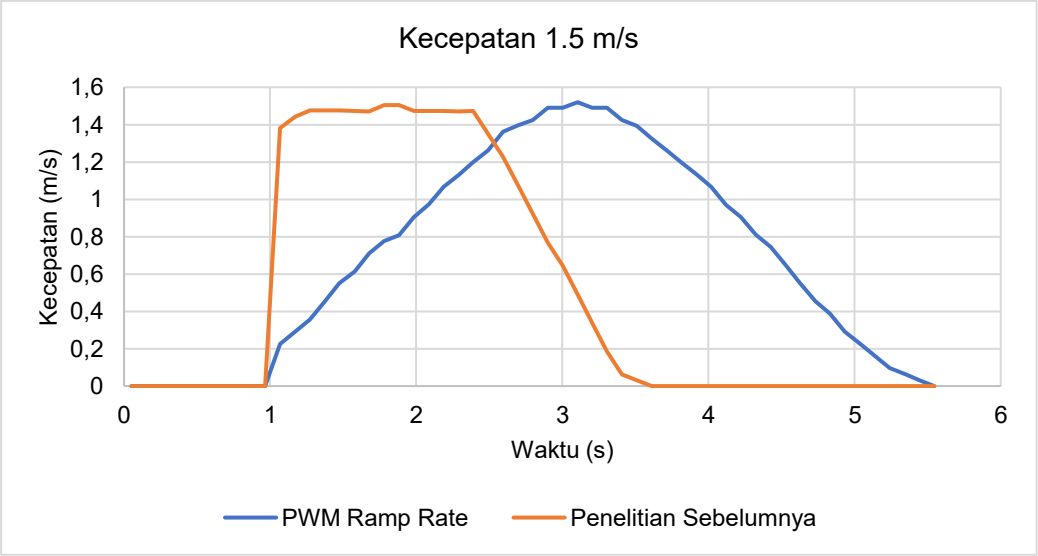
Gambar 21 Grafik Kecepatan Linear 0.22 m/s



Gambar 22 Grafik Kecepatan Linear 0.5 m/s



Gambar 23 Grafik Kecepatan Linear 1 m/s



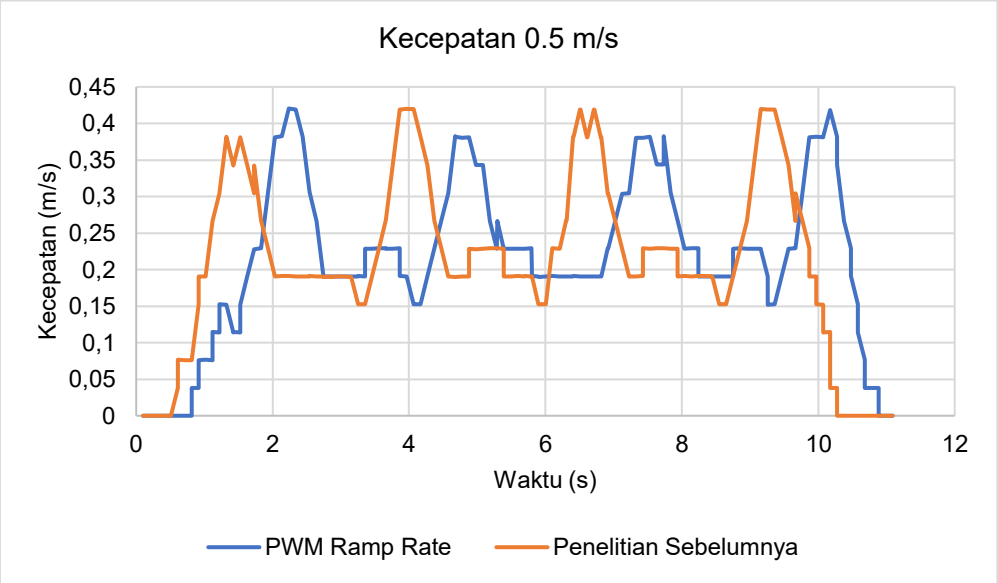
Gambar 24 Grafik Kecepatan Linear 1.5 m/s

Berdasarkan hasil perbandingan kecepatan PWM Ramp Rate dan penelitian sebelumnya, dengan pengujian jarak 2 meter diperoleh untuk kecepatan 0.22 m/s kontrol PWM Ramp Rate dan penelitian sebelumnya masing-masing membutuhkan waktu selama 29 detik untuk mencapai target. Untuk kecepatan 0.5 m/s, kontrol PWM Ramp Rate membutuhkan waktu 10.7 detik sedangkan penelitian sebelumnya membutuhkan waktu 9.3 detik. Pada kecepatan 1 m/s, kontrol PWM Ramp Rate membutuhkan waktu 6.4 detik sedangkan penelitian sebelumnya 5.4 detik dan untuk kecepatan 1.5 m/s, kontrol PWM Ramp Rate membutuhkan 5.5 detik sedangkan penelitian sebelumnya 3.6 detik dalam mencapai target

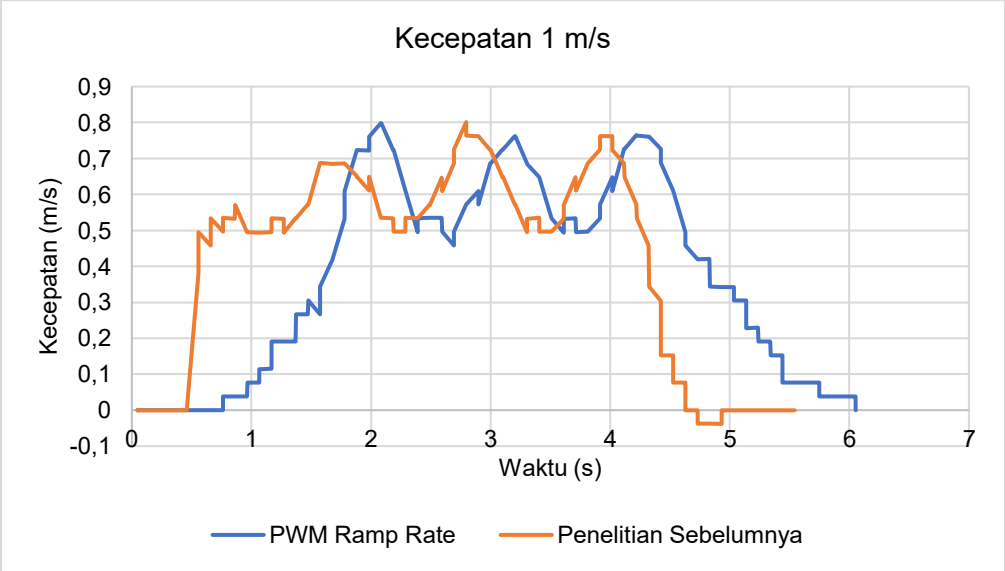
Pada kecepatan 0.22 m/s, waktu dan grafik yang diperoleh dengan kontrol PWM Ramp Rate dan Penelitian sebelumnya hampir sama dan sentakan hampir tidak terlihat. Mulai dari kecepatan 0.5 untuk hingga 1.5 m/s grafik penelitian sebelumnya terlihat

menjulang khususnya pada rise time dan fall time, menyebabkan Waiter-Bot mengalami sentakan sehingga pergerakannya kasar namun robot dengan cepat mampu mencapai target, sedangkan dengan menggunakan PWM Ramp Rate, robot perlahan meningkatkan kecepatannya hal ini menghasilkan pergerakan robot lebih halus namun membutuhkan waktu yang lebih lambat mencapai target.

Untuk kecepatan angular, dilakukan pengujian dengan parameter yang digunakan ialah 1 dan 0.5 m/s dalam melakukan 1 putaran penuh, sehingga diperoleh data berikut:



Gambar 25 Grafik Kecepatan Angular 0.5 m/s

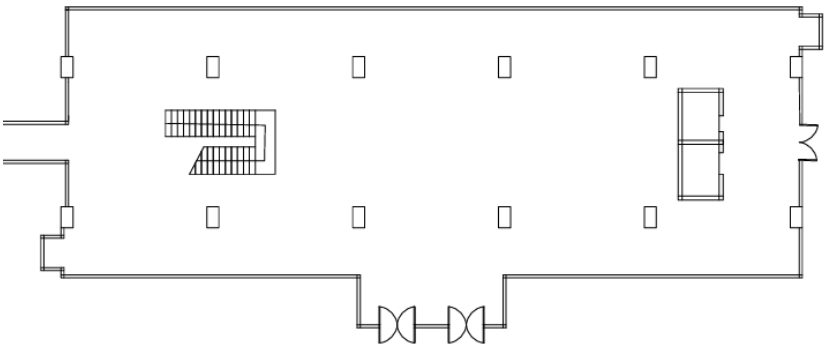


Gambar 26 Grafik Kecepatan Angular 1 m/s

.Berdasarkan hasil perbandingan kontrol PWM Ramp Rate dan penelitian sebelumnya pada kecepatan angular dengan pengujian 1 putaran penuh. Diperoleh untuk kecepatan 0.5 m/s, kontrol PWM Ramp Rate membutuhkan waktu 10.8 detik sedangkan penelitian sebelumnya membutuhkan waktu 10.2 detik. Pada kecepatan 1 m/s, kontrol PWM Ramp Rate membutuhkan waktu 6 detik sedangkan penelitian sebelumnya membutuhkan waktu 4.9 detik untuk 1 putaran penuh. Terlihat untuk kemampuan manuver waiter-bot menghasilkan pergerakan yang lebih halus dengan menggunakan PWM Ramp Rate jika dibandingkan penelitian sebelumnya namun membutuhkan waktu yang lebih lambat dalam mencapai target.

3.1.2 Uji Mapping

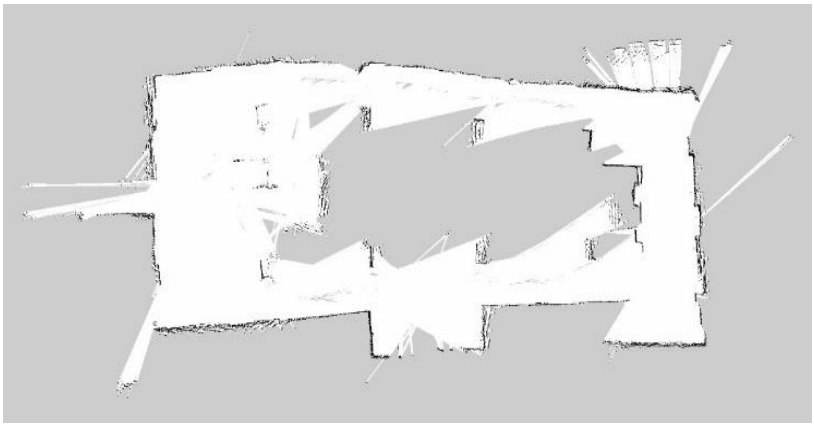
Pengujian hasil mapping dilakukan dengan membandingkan hasil mapping dengan setting penelitian kontrol motor konvensional (instan) dan kontrol motor yang telah ditingkatkan dengan PWM Ramp Rate pada ruangan dimensi panjang 36.8 m dan lebar 13.25 m.



Gambar 27 Denah Ruangan Pengujian

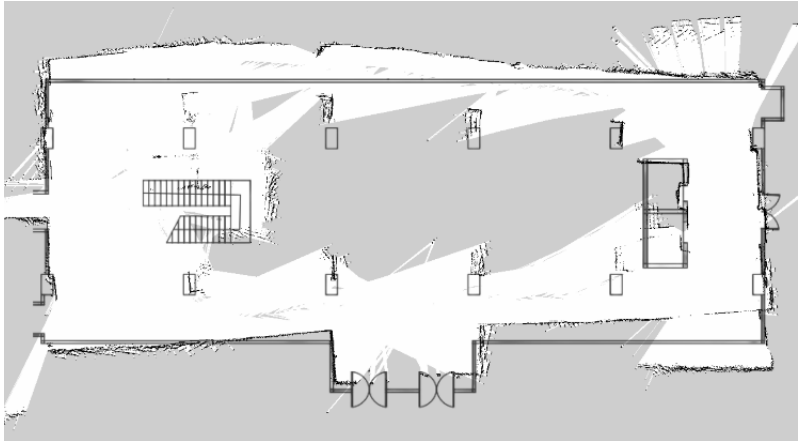
3.1.2.1 Hasil Mapping Dengan Setting Penelitian Sebelumnya

Adapun untuk hasil mapping dengan setting kontrol motor penelitian sebelumnya yaitu kontrol motor konvensional diperoleh



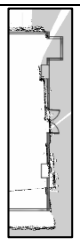
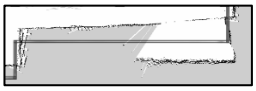
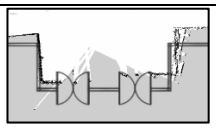
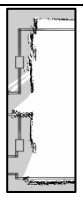
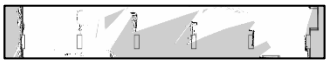
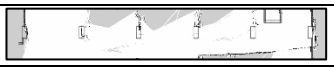
Gambar 28 Hasil Mapping Kontrol Motor Penelitian Sebelumnya

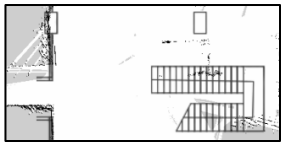
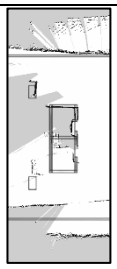
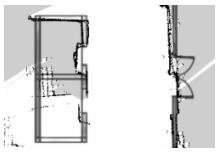
Dengan membandingkan Mapping dengan setting kontrol motor konvensional dan ground truth diperoleh data berikut.



Gambar 29 Perbandingan Mapping Setting Kontrol Motor Penelitian sebelumnya dan Ground Truth

Tabel 1 Hasil Mapping Setting Kontrol Motor Penelitian sebelumnya dan Ground Truth

No.	Map	Ground Truth	Hasil Pembacaan	Error (%)
1.		36.80	35,83	2.63%%
2.		13.25	11,90	10.19%
3.		14.82	22,32	50.64%
4.		12.15	12,16	0.09%
5.		13.25	14,69	10.85%
6.		35.90	34,23	59.09%
7.		35.90	34,53	3.81%

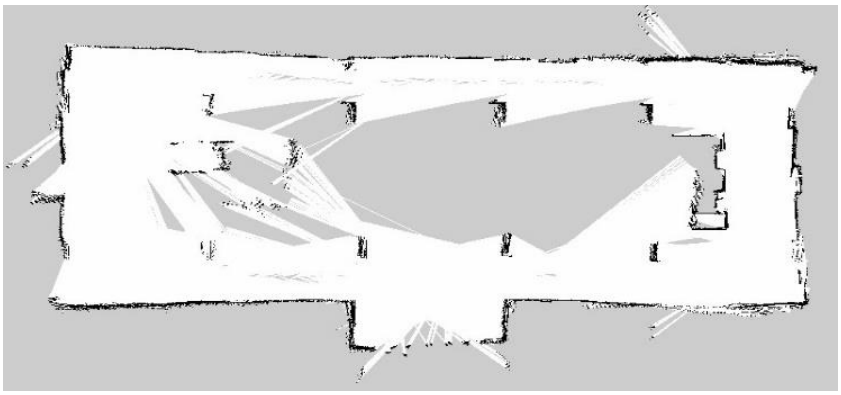
8.		13.25	14,55	9.79%
9.		4.85	5,04	4.02%
10.		13.25	14,04	5.99%
11.		3.82	3,40	11.06%

Rata – Rata Error 15.29%

Dengan menggunakan kontrol motor penelitian sebelumnya diperoleh rata-rata error sebesar 15.29%

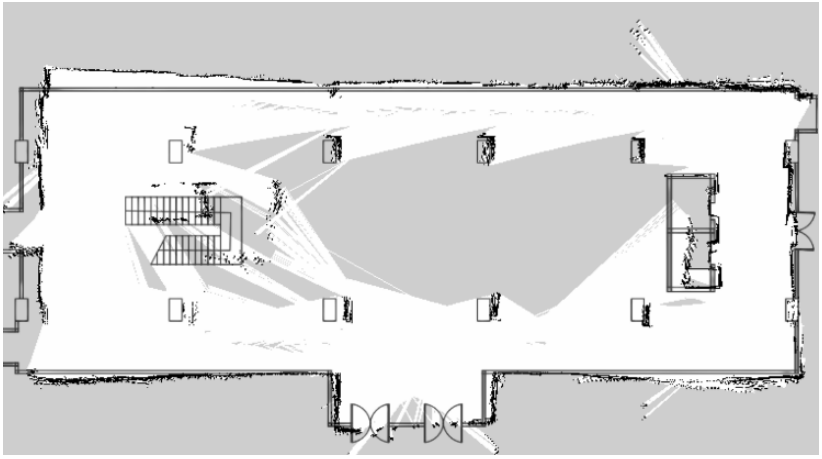
3.1.2.2 Hasil Mapping Dengan Setting Kontrol Motor PWM Ramp Rate

Adapun untuk hasil mapping dengan setting kontrol motor PWM Ramp Rate diperoleh



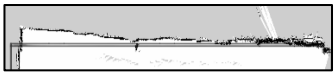
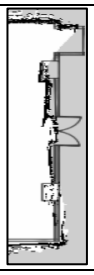
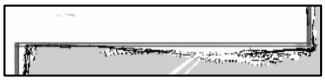
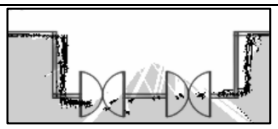
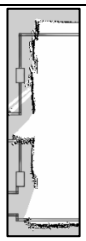
Gambar 30 Hasil Mapping Kontrol Motor PWM Ramp Rate

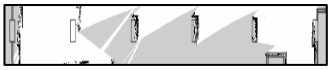
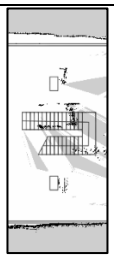
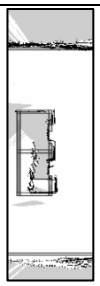
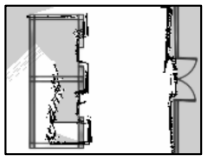
Dengan membandingkan Mapping dengan setting kontrol motor PWM Ramp Rate dan ground truth diperoleh data berikut.



Gambar 31 Perbandingan Mapping Setting Kontrol Motor PWM Ramp Rate dan Ground Truht

Tabel 2 Hasil Mapping Setting PWM Ramp Rate dan Ground Truth

No.	Map	Ground Truth	Hasil Pembacaan	Error (%)
1.		36.80	35.00	4.90%
2.		13.25	13.06	1.45%
3.		14.82	14.24	3.95%
4.		12.15	11.97	1.47%
5.		13.25	14.41	8.75%

6.		35.90	40.62	13.14%
7.		35.90	34.45	4.05%
8.		13.25	14.15	6.80%
9.		4.85	5.11	5.40%
10.		13.25	13.60	2.63%
11.		3.82	3.23	15.41%

Rata – Rata Error 6.18%

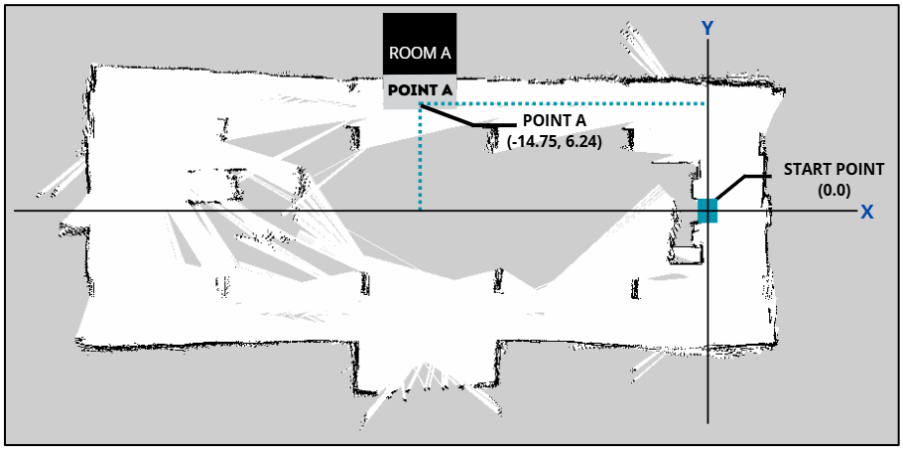
Hasil pengukuran dengan membandingkan ground truth dan hasil mapping Waiter-Bot dengan RVIZ, diperoleh peningkatan hasil maping kontrol motor PWM Ramp Rate lebih baik dengan error sebesar 6.18%. Sedangkan hasil mapping setting kontrol motor konvensional diperoleh error sebesar 15.29%

3.1.3 Uji Navigasi

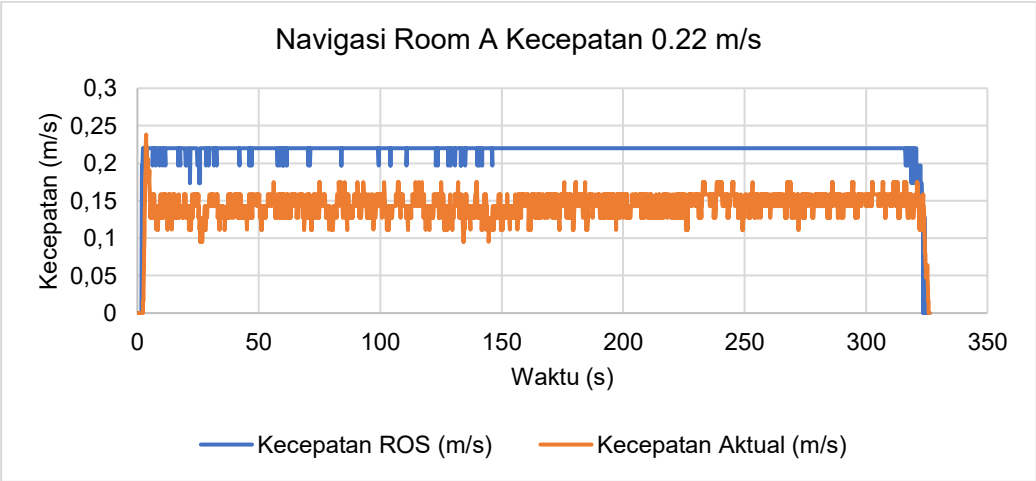
Pengujian dilakukan dengan 3 titik tujuan yang berbeda dan parameter kecepatan yang berbeda.

3.1.3.1 Navigasi Point A

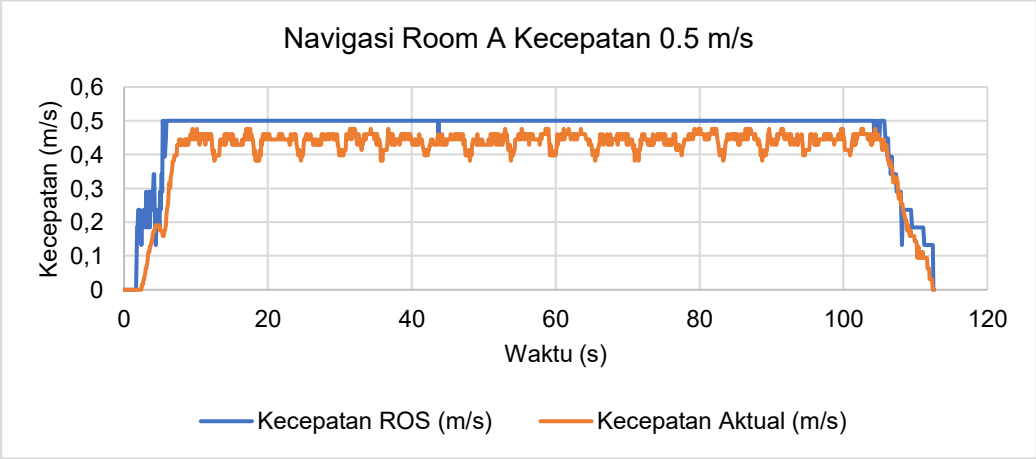
Pengujian Point A dilakukan dengan target kordinat x(-14.75) dan y(6.24) dari start point, diperoleh data berikut.



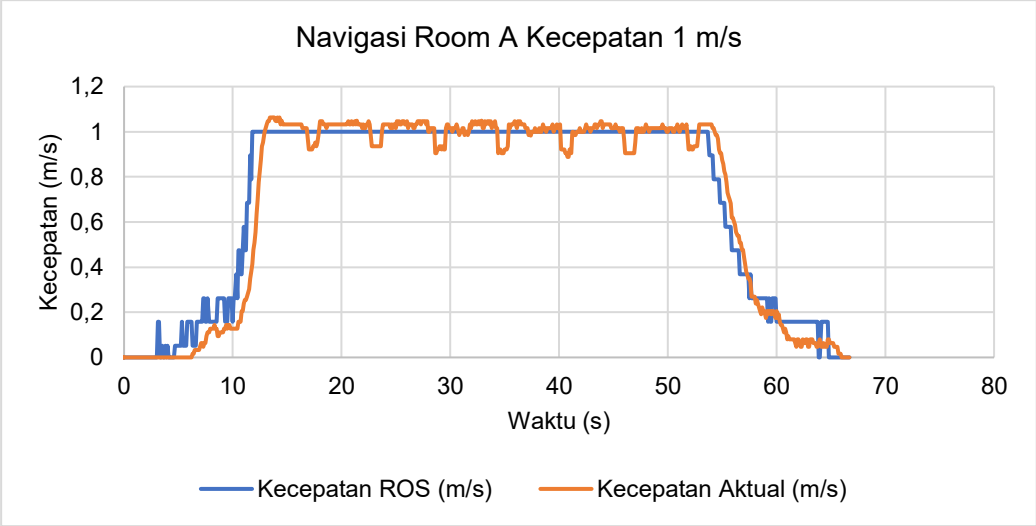
Gambar 32 Jarak Navigasi Point A



Gambar 33 Grafik Navigasi Point A Kecepatan 0.22 m/s



Gambar 34 Grafik Navigasi Point A Kecepatan 0.5 m/s

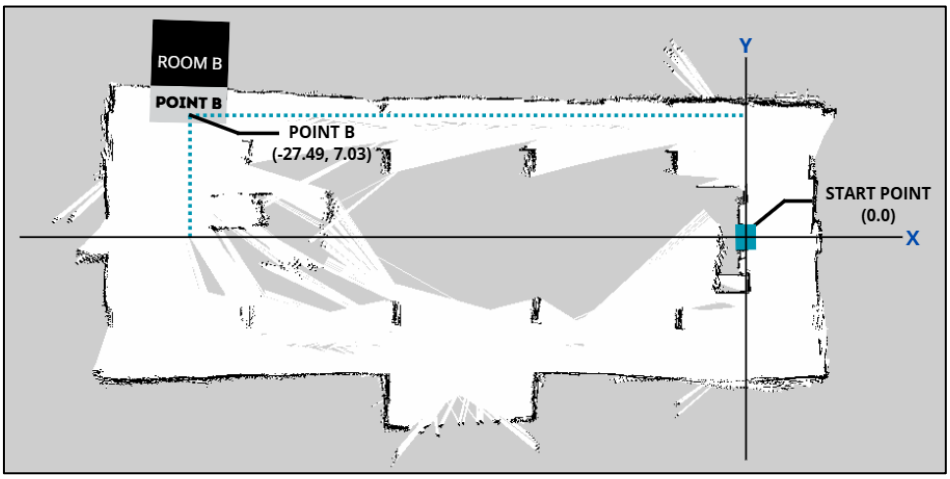


Gambar 35 Grafik Navigasi Point A Kecepatan 1 m/s

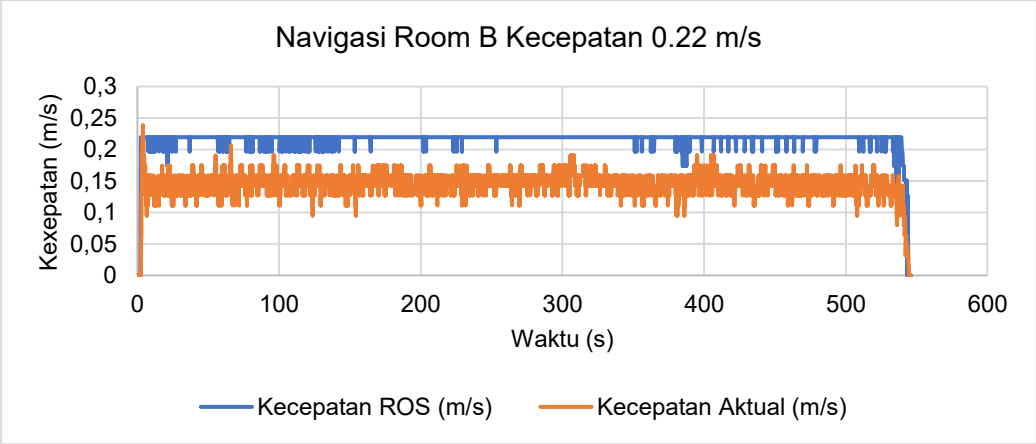
Dari pengujian navigasi Point A diperoleh hasil untuk kecepatan 0.22 m/s (penelitian sebelumnya) membutuhkan waktu 5 menit 25 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual yaitu 0.14 m/s. Untuk kecepatan 0.5 m/s membutuhkan 1 menit 52 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual 0.44 m/s. Terakhir untuk kecepatan 1 m/s membutuhkan waktu 1 menit 6 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual 1 m.s

3.1.3.2 Navigasi Point B

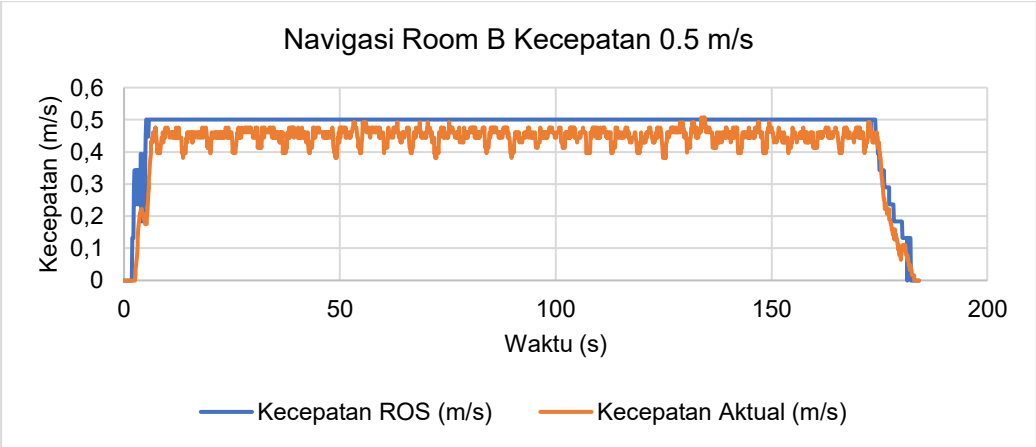
Pengujian Point B dilakukan dengan tujuan titik kordinat x(-27.49) dan y(7.03) dari start point, diperoleh data berikut.



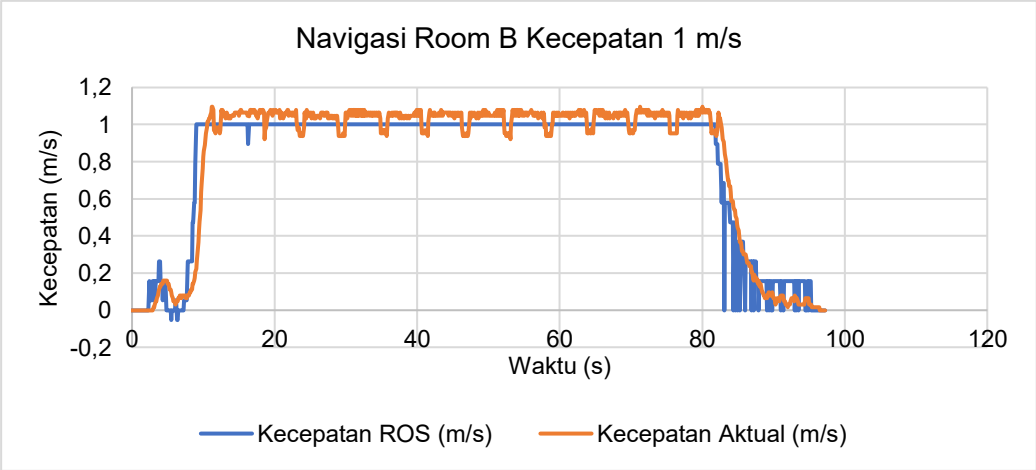
Gambar 36 Jarak Navigasi Point B



Gambar 37 Grafik Navigasi Point B Kecepatan 0.22 m/s



Gambar 38 Grafik Navigasi Point B Kecepatan 0.5 m/s

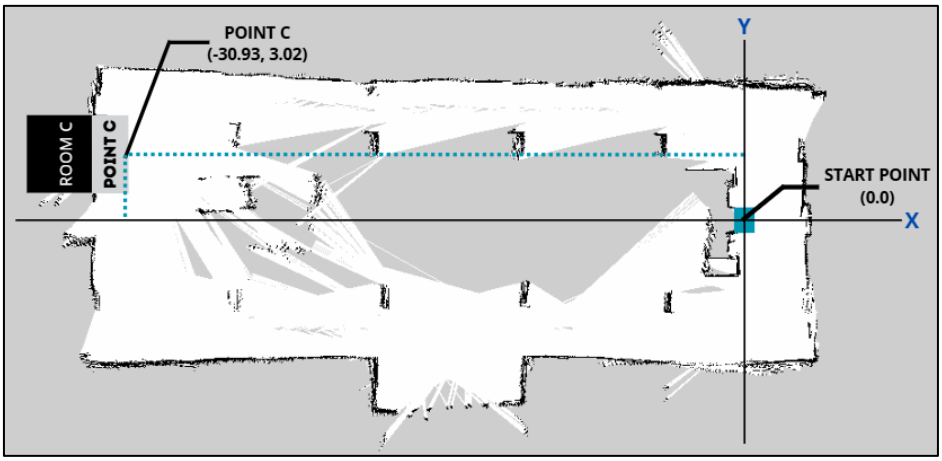


Gambar 39 Grafik Navigasi Point B Kecepatan 1 m/s

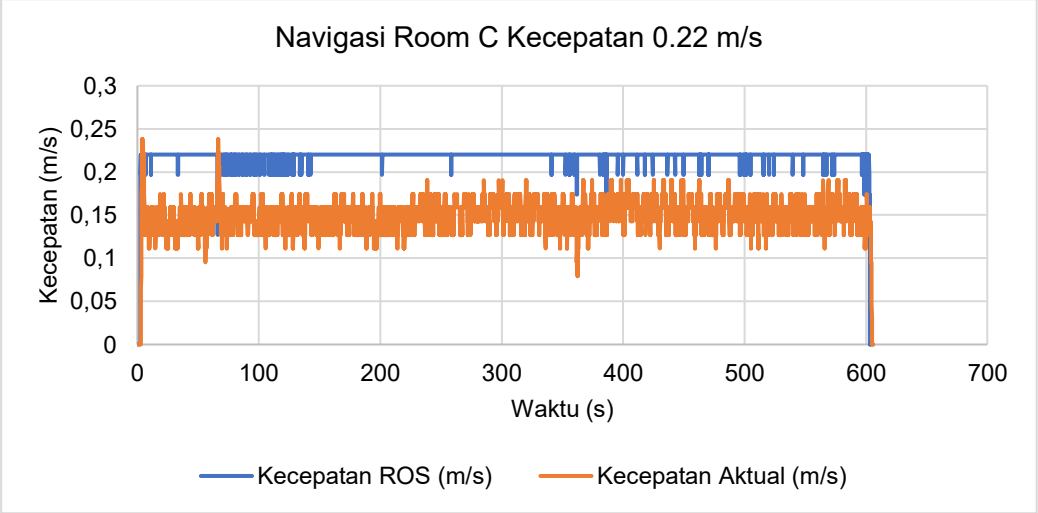
Dari pengujian navigasi Point B diperoleh hasil untuk kecepatan 0.22 m/s (penelitian sebelumnya) membutuhkan waktu 9 menit 4 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual yaitu 0.14 m/s. Untuk kecepatan 0.5 m/s membutuhkan 3 menit 3 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual 0.45 m/s. Terakhir untuk kecepatan 1 m/s membutuhkan waktu 1 menit 36 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual 1.03 m/s

3.1.3.3 Navigasi Point C

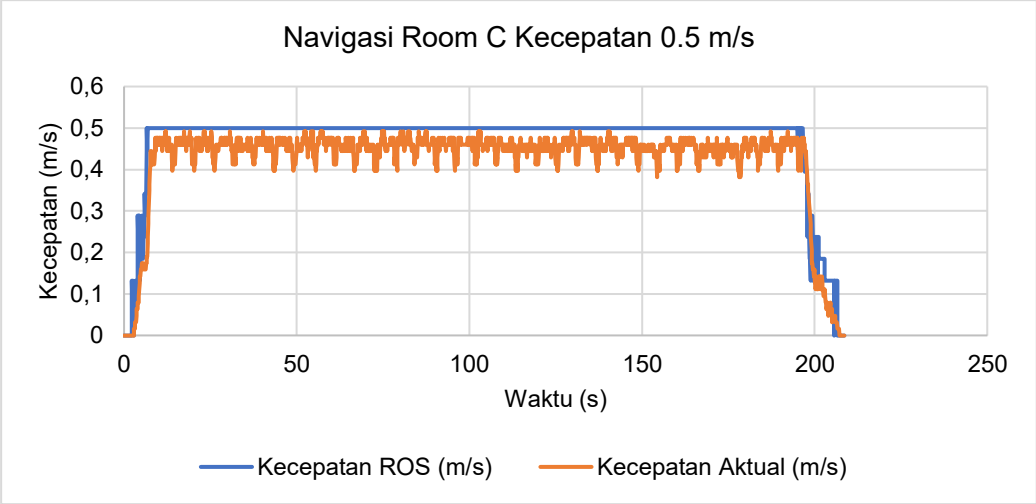
Pengujian Point C dilakukan dengan tujuan titik kordinat x(-30.93) dan y(3.02) dari start point, diperoleh data berikut.



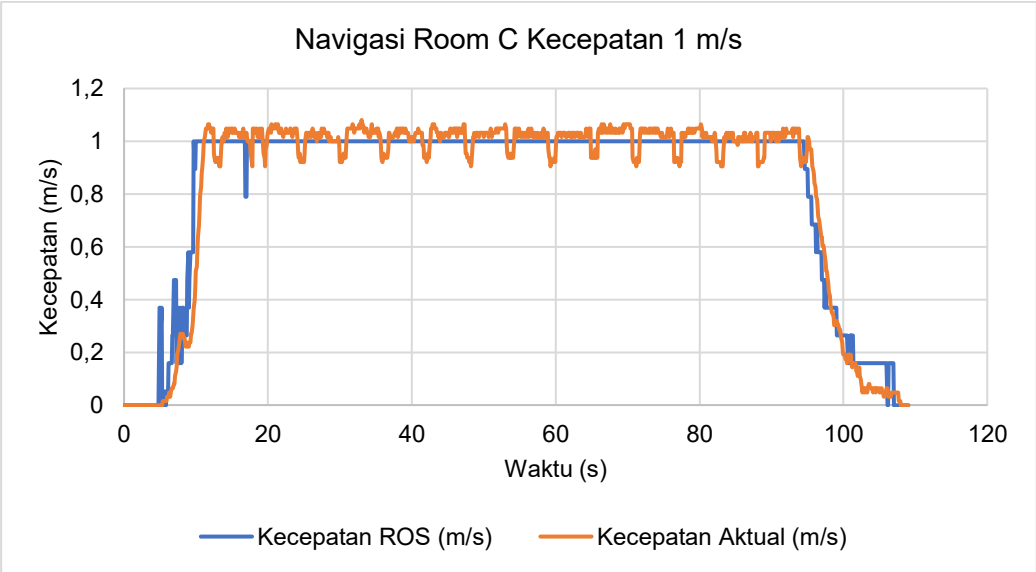
Gambar 40 Jarak Navigasi Point C



Gambar 41 Grafik Navigasi Point C Kecepatan 0.22 m/s



Gambar 42 Grafik Navigasi Point C Kecepatan 0.5 m/s

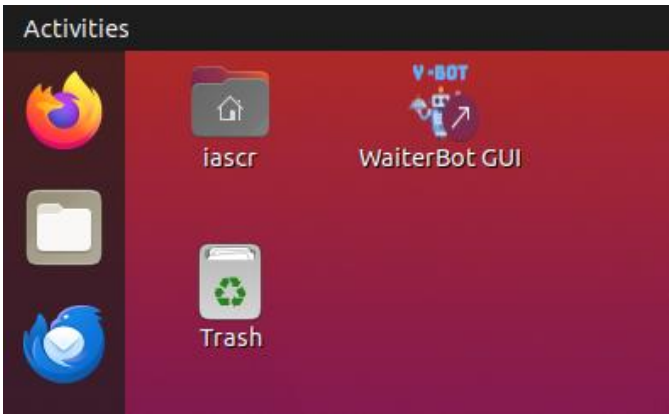


Gambar 43 Grafik Navigasi Point C Kecepatan 1 m/s

Dari pengujian navigasi Point C diperoleh hasil untuk kecepatan 0.22 m/s (penelitian sebelumnya) membutuhkan waktu 10 menit 5 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual yaitu 0.15 m/s. Untuk kecepatan 0.5 m/s membutuhkan 3 menit 27 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual 0.45 m/s. Terakhir untuk kecepatan 1 m/s membutuhkan waktu 1 menit 47 detik untuk mencapai tujuan dengan rata-rata kecepatan aktual 1.01 m/s

3.2 Integrasi Graphical User Interface (GUI) dengan Kivy

Berdasarkan hasil integrasi GUI, diketahui bahwa kivy dapat di implementasikan dengan ROS pada Y-Bot dengan baik yang memungkinkan pengoperasian tanpa terminal.



Gambar 44 Aplikasi Interface Y-Bot Pada Desktop

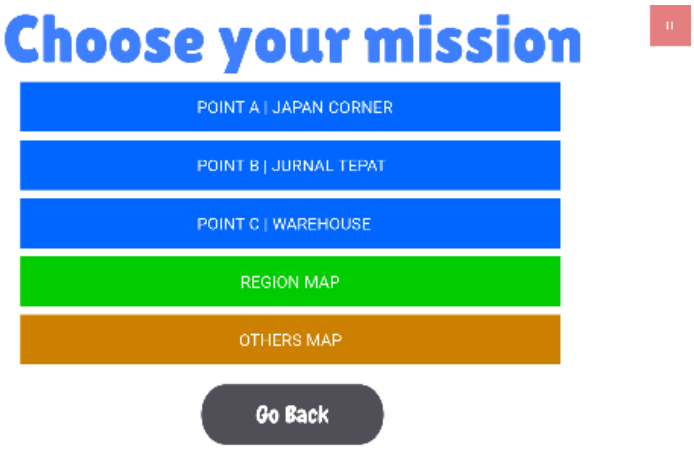
Gambar 45 Menu Awal

Gambar 46 Menu Multi-Mode

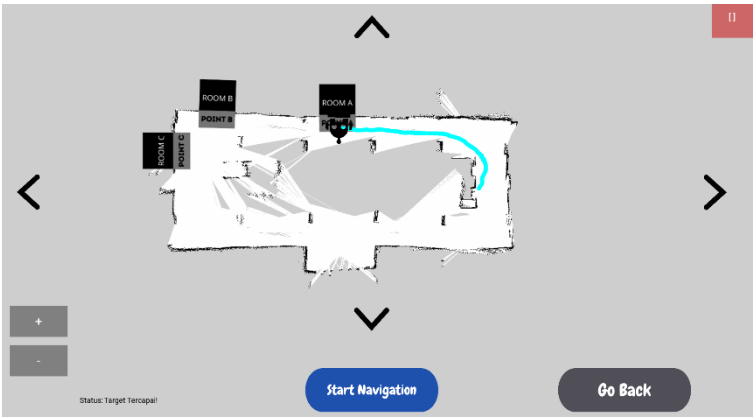
Gambar 47 Mode Controller (Joystick)

Gambar 48 Penamaan Mode Mapping

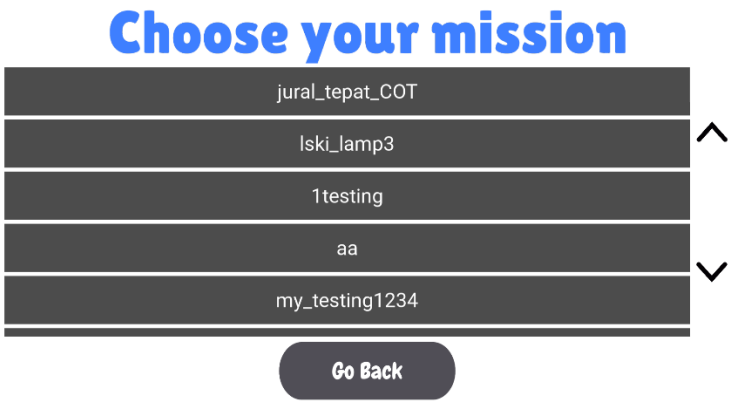
Gambar 49 Mode Mapping



Gambar 50 Menu Misi Navigasi



Gambar 51 Mode Navigasi



Gambar 52 Menu Others Map Mode Navigasi

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan hasil dan pembahasan dari bab sebelumnya, maka dapat diperoleh kesimpulan,

1. Uji kecepatan linear dan angular, berdasarkan hasil penelitian diperoleh bahwa pergerakan motor akan menimbulkan sentakan pada kecepatan tinggi yang menyebabkan robot mengalami pergerakan yang kasar, sedangkan dengan kontrol PWM Ramp Rate pergerakan motor lebih halus pada saat robot bergerak dan bermanuver khususnya pada kecepatan tinggi,
2. Uji mapping pada ruangan dengan dimensi panjang 36.8 meter dan lebar 13.25 meter diperoleh error rata-rata yang lebih rendah dengan hasil 6.18%, sedangkan untuk hasil mapping setting penelitian sebelumnya diperoleh error sebesar 15.29%, hal ini menunjukkan dengan pergerakan robot yang lebih halus menghasilkan peta yang lebih optimal terhadap ground truth,
3. Uji navigasi yang dilakukan pada 3 Point yaitu A,B dan C. Untuk kecepatan 0.22 m/s (setting penelitian sebelumnya) diperoleh hasil waktu tempuh navigasi untuk masing-masing tujuan A,B,C adalah 5 menit 25 detik, 9 menit 4 detik, dan 10 menit 5 detik, perintah kecepatan dari ROS tidak dapat digapai dengan perolehan rata-rata kecepatan 0.14 m/s. Untuk kecepatan 0.5 m/s diperoleh hasil waktu tempuh navigasi untuk masing-masing tujuan A,B, C yaitu 1 menit 52 detik, 3 menit 3 detik, 3 menit 27 detik, kecepatan aktual mendekati perintah kecepatan dari ROS dengan rata-rata kecepatan 0.45 m/s. Untuk kecepatan 1 m/s diperoleh hasil waktu tempuh navigasi untuk masing-masing tujuan A, B, C ialah 1 menit 6 detik, 1 menit 36 detik, dan 1 menit 47 detik, hasil memunjukkan kecepatan aktual menyamakan perintah kecepatan dari ROS dengan rata-rata kecepatan 1 m/s. Sehingga pada pengujian ini didapatkan kualitas pergerakan Waiter-Bot meningkat dengan menggunakan PWM Ramp Rate.
4. Implementasi GUI berbasis Kivy berhasil terintegrasi dengan ROS, memungkinkan pengoperasian waiter-bot dengan kontrol manual, pemetaan, dan navigasi dalam satu dashboard terpusat, sehingga meniadakan kebutuhan input berbasi terminal.

4.2 Saran

Berdasarkan hasil penelitian, disarankan agar pengembangan sistem dengan metode yang lebih canggih untuk hasil mapping dan navigasi yang lebih akurat. Selain

itu, dapat digunakan hardware atau sensor yang lebih mutahkhir dalam melakukan pemetaan maupun navigasi seperti sensor lidar atau sensor kinect yang terbarukan.

DAFTAR PUSTAKA

Anshar, M. (2025). *DASAR PEMROGRAMAN MICROPYTHON DAN APLIKASI PRAKTIS PADA ROBOT CERDAS*. Greenbook Publishing Indonesia.

Ben Shneiderman, C. P. (2016). *Designing the User Interface: Strategies for Effective Human-Computer Interaction, 6th ed. Pearson*.

Borenstein, J., & Feng, L. (1996). *Measurement and Correction of Systematic Odometry Errors in Mobile Robots*. *12*(5).

Bruno Siciliano, O. K. (2016). *Springer Hand book of Robotics*. https://doi.org/10.1007/978-3-319-32552-1

Faizal, F. A., Anshar, M., Salam, A. E. U., & Aiman, M. (2025). *RANCANG BANGUN SISTEM PEMETAAN DAN LOKALISASI BERBASIS A LGORITMA SLAM MENGGUNAKAN DEPTH SENSOR KINECT PADA MOBIL E ROBOT*. *13*(3).

Gonzalez-aguirre, J. A., Osorio-oliveros, R., Rodríguez-hernández, K. L., Lizárraga-iturralde, J., Menendez, R. M., Ramírez-mendoza, R. A., Ramírez-moreno, M. A., & Lozoya-santos, J. D. J. (2021). *applied sciences Service Robots : Trends and Technology*.

Jalil, A. (2018). *ROBOT OPERATING SYSTEM (ROS) DAN GAZEBO SEBAGAI MEDIA PEMBELAJARAN ROBOT INTERAKTIF*. *10*, 284–289.

Joseph, L. (2015). *Mastering ROS for Robotics Programming*.

Kane, J. M. O., & Kane, J. M. O. (2018). *A Gentle Introduction to ROS*.

Lundh, H. (2022). *Generating user interfaces for ROS-based robots*.

Lynch, K. M., & Park, F. C. (2017). *MODERN ROBOTICS MECHANICS , PLANNING , AND CONTROL* (Issue May).

Prayoga, A., & Kurniawan, B. (2018). Prototype Robot Waiter at Fast Food Restaurants. *Telekontran : Jurnal Ilmiah Telekomunikasi, Kendali Dan Elektronika Terapan*, *6*(2), 80–91. https://doi.org/10.34010/telekontran.v6i2.3802

Quigley, M., Gerkey, B., Conley, K., Faust, J., Foote, T., Leibs, J., Berger, E., Wheeler, R., & Ng, A. (n.d.). *ROS : an open-source Robot Operating System*. *Figure 1*.

Salam, A. (2024). *RANCANG BANGUN SISTEM NAVIGASI ROBOT OTONOM WAITER BOT BERBASIS ROBOT OPERATING SYSTEM*.

Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots second edition*.

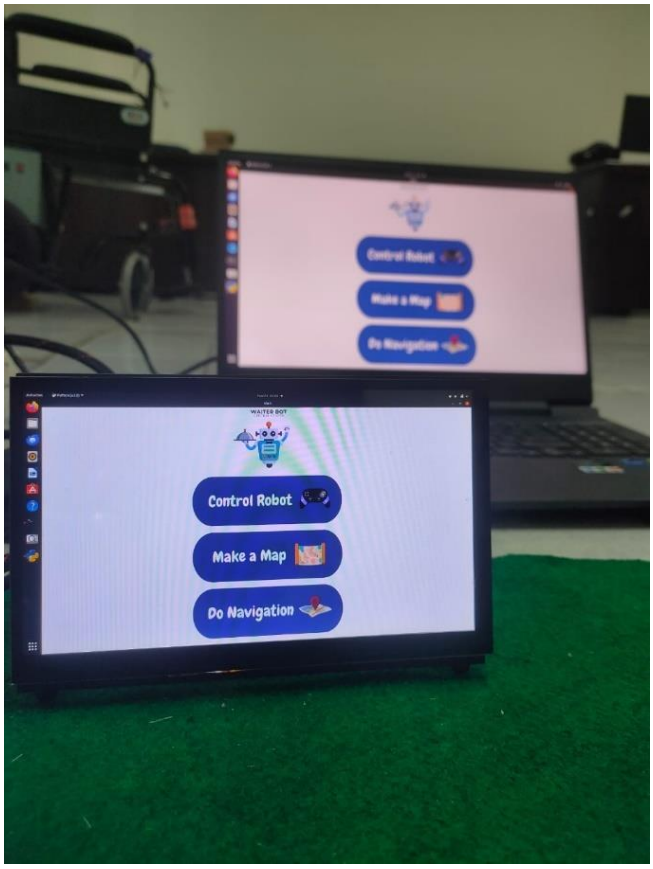
Singla, D. (2023). Developing Online Application with Kivy : A Python Framework. *International Journal of Trend in Scientific Research and Development (IJTSRD)*, *7*(4), 504–509.

Yoga Prihastomo, & Winanti. (2024). TREN PERKEMBANGAN GRAPHICAL USER INTERFACE MELALUI PERMOHONAN DESAIN INDUSTRI DI INDONESIA Yoga. *JOCE IP*, *18*(2).

LAMPIRAN

Lampiran 1

Pemasangan dan desain interface



Lampiran 2

gui.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# 1. KONFIGURASI SISTEM & LIBRARY
# Mengatur konfigurasi input Kivy agar support layar sentuh dan keyboard dock
from kivy.config import Config
Config.set('kivy', 'keyboard_mode', 'systemanddock')
if Config.has_section('input'):
    for key, value in Config.items('input'):
        Config.remove_option('input', key)
Config.set('input', 'mouse', 'mouse,disable_multitouch')

# Import modul-modul UI (User Interface) dari Kivy
from kivy.app import App
from kivy.lang import Builder
from kivy.uix.screenmanager import Screen
from kivy.uix.button import Button
from kivy.uix.label import Label
from kivy.clock import mainthread, Clock
from functools import partial
from kivy.uix.image import Image
from kivy.uix.behaviors import TouchRippleBehavior, ButtonBehavior
from kivy.properties import ObjectProperty, NumericProperty, BooleanProperty
from kivy.core.window import Window
from kivy.uix.widget import Widget
from kivy.core.audio import SoundLoader
from kivy.graphics import Color, Line # Library untuk menggambar garis jalur robot

# Import modul sistem operasi dan perhitungan
import yaml
```

```
import math
import os
import subprocess
import threading
from manager import RosManager

# ==================================================================
# 2. SCREEN: MENU PEMILIHAN PETA (NAV SELECTION)
# ==================================================================
class NavSelectionScreen(Screen):
    def on_enter(self):
        # Saat layar masuk, langsung muat menu utama
        self.show_main_menu()

    def show_main_menu(self):
        """Menampilkan tombol-tombol preset (A, B, C) dan daftar peta."""
        grid = self.ids.nav_map_grid
        grid.clear_widgets()
        app = App.get_running_app()

        # Reset tampilan scroll (sembunyikan scrollbar di menu awal)
        if 'scroll_container' in self.ids:
            self.ids.scroll_container.width = 0
            self.ids.scroll_container.opacity = 0
        if 'btn_scroll_up' in self.ids: self.ids.btn_scroll_up.disabled = True
        if 'btn_scroll_down' in self.ids: self.ids.btn_scroll_down.disabled = True

        # Konfigurasi ukuran tombol
```

```
BTN_WIDTH = '1300dp'
BTN_HEIGHT = '120dp'
FONT_SIZE = '40sp'

# Fungsi pembantu untuk membuat tombol seragam
def create_menu_btn(text, color, action, align_mode, padding_val):
    btn = Button(
        text=text,
        size_hint=(None, None),
        width=BTN_WIDTH,
        height=BTN_HEIGHT,
        pos_hint={'center_x': 0.5},
        font_size=FONT_SIZE,
        background_color=color,
        halign=align_mode,
        valign='middle',
        padding_x=padding_val
    )
    def update_text_size(instance, value):
        instance.text_size = (instance.width, None)
    btn.bind(size=update_text_size)
    btn.text_size = (530, None)
    btn.bind(on_press=action)
    return btn

PADDING_POINT = 1300

# --- Tombol Preset A, B, C ---
grid.add_widget(create_menu_btn(
    "    POINT A | JAPAN CORNER", (0, 0.4, 1, 1),
    partial(app.start_preset_navigation, 'A'),
```

```
        'left', PADDING_POINT
    ))

    grid.add_widget(create_menu_btn(
        "    POINT B | JURNAL TEPAT", (0, 0.4, 1, 1),
        partial(app.start_preset_navigation, 'B'),
        'left', PADDING_POINT
    ))

    grid.add_widget(create_menu_btn(
        "    POINT C | WAREHOUSE", (0, 0.4, 1, 1),
        partial(app.start_preset_navigation, 'C'),
        'left', PADDING_POINT
    ))

    # --- Tombol Region Map ---
    grid.add_widget(create_menu_btn(
        "REGION MAP", (0, 0.8, 0, 1),
        partial(app.start_navigation_with_map, 'test1','None'),
        'center', 0
    ))

    # --- Tombol Others Map (Menu Lanjutan) ---
    others_btn = create_menu_btn(
        "OTHERS MAP", (0.8, 0.5, 0, 1),
        self.go_to_others_map_with_audio,
        'center', 0
    )
    grid.add_widget(others_btn)

def go_to_others_map_with_audio(self, instance=None):
```

```
        """Memainkan audio menu others lalu menampilkan daftar peta."""
        app = App.get_running_app()
        app.play_audio('others_map.mp3')
        Clock.schedule_once(lambda dt: self.show_others_map(), 0.2)

    def show_others_map(self, instance=None):
        """Membaca folder peta secara manual dan menampilkan listnya."""
        grid = self.ids.nav_map_grid
        grid.clear_widgets()
        app = App.get_running_app()

        # Aktifkan scrollbar UI
        if 'scroll_container' in self.ids:
            self.ids.scroll_container.width = 100
            self.ids.scroll_container.opacity = 1
        if 'btn_scroll_up' in self.ids: self.ids.btn_scroll_up.disabled = False
        if 'btn_scroll_down' in self.ids: self.ids.btn_scroll_down.disabled = False

        # Tombol Kembali
        btn_back = Button(
            text="[ KEMBALI KE MENU UTAMA ]",
            size_hint_y=None, height='80dp',
            size_hint_x=0.9, pos_hint={'center_x': 0.5},
            font_size='25sp', background_color=(1, 0, 0, 1)
        )
        btn_back.bind(on_press=lambda x: self.show_main_menu())
        grid.add_widget(btn_back)

        # 1. Tentukan folder map secara manual
        maps_folder =
os.path.expanduser("~/catkin_ws/src/autonomus_mobile_robot/maps")
```

```
    map_names = []

    # 2. Scan folder tersebut
    if os.path.exists(maps_folder):
        try:
            files = os.listdir(maps_folder)
            map_names = [f.replace('.yaml', '') for f in files if f.endswith('.yaml')]
            map_names.sort()
        except Exception as e:
            print(f"Error reading maps: {e}")
    else:
        print(f"WARNING: Folder {maps_folder} tidak ditemukan.")

    if not map_names:
        grid.add_widget(Label(text="Tidak ada peta ditemukan.", color=(0,0,0,1)))
        return

    # 3. Buat tombol untuk setiap peta
    for name in map_names:
        btn = Button(
            text=name,
            size_hint_y=None, height='120dp',
            font_size='40sp'
        )
        btn.bind(on_press=partial(app.start_navigation_with_map, name))
        grid.add_widget(btn)

# ==================================================================
# 3. WIDGET KOSTUM (Custom Widgets)
```

```
#
==================================================================
===========
class ImageButton(ButtonBehavior, Image):
    pass

class MapImage(TouchRippleBehavior, Image):
    """Widget Peta yang bisa di-klik untuk set goal."""
    marker = ObjectProperty(None, allownone=True)
    locked = BooleanProperty(False)

    def on_touch_down(self, touch):
        if self.collide_point(*touch.pos):
            if self.locked:
                return False

            if self.marker and self.marker.parent:
                self.remove_widget(self.marker)

            # Buat marker visual 'X'
            new_marker = Label(text='X', font_size='30sp', color=(1, 0, 0, 1), bold=True)
            new_marker.center = touch.pos
            self.add_widget(new_marker)
            self.marker = new_marker

            # Hitung koordinat dan kirim ke App
            self.save_map_coords_to_marker(touch, new_marker)
            App.get_running_app().calculate_ros_goal(touch, self)
            return super().on_touch_down(touch)
        return False

    def save_map_coords_to_marker(self, touch, marker_widget):
```

```
"""Konversi dari Pixel Layar ke Koordinat Peta (Meter)."""
app = App.get_running_app()
if not app.manager.map_metadata: return

meta = app.manager.map_metadata
resolution = meta['resolution']
origin_x = meta['origin'][0]
origin_y = meta['origin'][1]

norm_w, norm_h = self.texture.size
widget_w, widget_h = self.size
if norm_w == 0 or norm_h == 0: return

img_ratio = norm_w / norm_h
widget_ratio = widget_w / widget_h

if widget_ratio > img_ratio:
    scale = widget_h / norm_h
    offset_x = (widget_w - norm_w * scale) / 2.0
    offset_y = 0.0
else:
    scale = widget_w / norm_w
    offset_x = 0.0
    offset_y = (widget_h - norm_h * scale) / 2.0

touch_on_image_x = touch.pos[0] - self.x - offset_x
touch_on_image_y = touch.pos[1] - self.y - offset_y

pixel_x = touch_on_image_x / scale
pixel_y = touch_on_image_y / scale
```

```
        map_x = (pixel_x * resolution) + origin_x
        map_y = (pixel_y * resolution) + origin_y

        marker_widget.map_coords = (map_x, map_y)

class RobotMarker(Image):
    angle = NumericProperty(0)

class HomeScreen(Screen):
    pass

#
==================================================================
===========
# 4. SCREEN NAVIGASI (TAMPILAN PETA UTAMA)
#
==================================================================
===========
class NavigationScreen(Screen):
    selected_goal_coords = None
    robot_marker = ObjectProperty(None, allownone=True)
    pending_preset_target = None
    use_image_marker = BooleanProperty(False)
    path_line = None # Variabel untuk garis jalur robot

    def on_enter(self):
        """Dipanggil saat layar navigasi muncul."""
        app = App.get_running_app()
        self.load_map_image(app.manager.current_map_name)

        map_viewer = self.ids.map_viewer
        if map_viewer.marker and map_viewer.marker.parent:
```

```
        map_viewer.remove_widget(map_viewer.marker)
        map_viewer.marker = None

    scatter = self.ids.scatter_map
    scatter.scale = 1.0
    scatter.pos = self.ids.map_container.pos

    # [FITUR TRAJECTORY] Inisialisasi garis (Line)
    if not self.path_line:
        with scatter.canvas:
            Color(0, 1, 1, 1) # Warna Cyan
            self.path_line = Line(points=[], width=2)

    # Inisialisasi Marker Robot (Panah)
    if not self.robot_marker:
        source = 'robot_arrow.png' if os.path.exists('robot_arrow.png') else
'atlas://data/images/defaulttheme/checkbox_on'
        self.robot_marker = RobotMarker(source=source, size_hint=(None, None),
size=(30, 30), allow_stretch=True, opacity=0)
        self.ids.scatter_map.add_widget(self.robot_marker)

    if self.pending_preset_target:
        Clock.schedule_once(lambda dt:
self.setup_preset_mode(self.pending_preset_target), 0)
    else:
        self.setup_manual_mode()

    map_viewer.bind(size=self.update_marker_position,
pos=self.update_marker_position)
    app.set_dpad_visibility(False)
    self.update_event = Clock.schedule_interval(self.update_robot_display, 0.1)
```

```
    def clear_path(self):
        """Menghapus garis jalur lama."""
        if self.path_line:
            self.path_line.points = []

    def setup_manual_mode(self):
        self.ids.map_viewer.locked = False
        self.use_image_marker = False
        self.selected_goal_coords = None
        self.ids.navigate_button.disabled = True
        self.ids.navigation_status_label.text = "Status: Pilih titik di peta"

    def setup_preset_mode(self, target_data):
        x, y, name = target_data
        self.ids.map_viewer.locked = True
        self.use_image_marker = True

        self.selected_goal_coords = (x, y)
        self.ids.navigate_button.disabled = False
        self.ids.navigation_status_label.text = f"Tujuan: Point {name}\nTekan START untuk
jalan."
        self.show_goal_marker(x, y)

    def on_leave(self):
        if hasattr(self, 'update_event'):
            self.update_event.cancel()
        if self.robot_marker:
            self.robot_marker.opacity = 0
        self.ids.map_viewer.unbind(size=self.update_marker_position,
pos=self.update_marker_position)
        self.pending_preset_target = None
        self.clear_path()
```

```
def update_marker_position(self, *args):
    map_viewer = self.ids.map_viewer
    if map_viewer.marker and hasattr(map_viewer.marker, 'map_coords'):
        map_x, map_y = map_viewer.marker.map_coords
        screen_pos = self.calculate_screen_pos(map_x, map_y)
        if screen_pos:
            map_viewer.marker.center = screen_pos

def calculate_screen_pos(self, map_x, map_y):
    app = App.get_running_app()
    map_viewer = self.ids.map_viewer
    if not app.manager.map_metadata or not map_viewer.texture: return None

    meta = app.manager.map_metadata
    resolution = meta['resolution']
    origin_x = meta['origin'][0]
    origin_y = meta['origin'][1]

    pixel_x = (map_x - origin_x) / resolution
    pixel_y = (map_y - origin_y) / resolution

    norm_w, norm_h = map_viewer.texture.size
    widget_w, widget_h = map_viewer.size
    if norm_w == 0 or norm_h == 0: return None

    img_ratio = norm_w / norm_h
    widget_ratio = widget_w / widget_h

    if widget_ratio > img_ratio:
        scale = widget_h / norm_h
```

```
            offset_x = (widget_w - norm_w * scale) / 2.0
            offset_y = 0.0
        else:
            scale = widget_w / norm_w
            offset_x = 0.0
            offset_y = (widget_h - norm_h * scale) / 2.0

        screen_x = (pixel_x * scale) + offset_x + map_viewer.x
        screen_y = (pixel_y * scale) + offset_y + map_viewer.y
        return (screen_x, screen_y)

    def show_goal_marker(self, map_x, map_y):
        app = App.get_running_app()
        map_viewer = self.ids.map_viewer

        if (not map_viewer.texture or map_viewer.texture.size[0] <= 1 or not app.manager.map_metadata):
            Clock.schedule_once(lambda dt: self.show_goal_marker(map_x, map_y), 0.5)
            return

        if map_viewer.marker and map_viewer.marker.parent:
            map_viewer.remove_widget(map_viewer.marker)

        screen_pos = self.calculate_screen_pos(map_x, map_y)
        if not screen_pos: return

        if self.use_image_marker:
            source = 'goals.png' if os.path.exists('goals.png') else 'atlas://data/images/defaulttheme/filechooser_folder'
            new_marker = Image(source=source, size_hint=(None, None), size=('35dp', '35dp'), allow_stretch=True)
        else:
```

```
        new_marker = Label(text='X', font_size='30sp', color=(1, 0, 0, 1), bold=True)

    new_marker.center = screen_pos
    new_marker.map_coords = (map_x, map_y)
    map_viewer.add_widget(new_marker)
    map_viewer.marker = new_marker

def load_map_image(self, map_name):
    if map_name:
        app = App.get_running_app()
        map_image_path = app.manager.get_map_image_path(map_name)
        if map_image_path:
            self.ids.map_viewer.source = map_image_path
            app.manager.load_map_metadata(map_name)
            self.ids.map_viewer.reload()

@mainthread
def update_robot_display(self, dt):
    """Update posisi robot di layar dan gambar jalur (path)."""
    app = App.get_running_app()
    pose = app.manager.get_robot_pose()
    if pose is None: return
    screen_pos = self.calculate_screen_pos(pose['x'], pose['y'])
    if screen_pos:
        self.robot_marker.opacity = 1
        self.robot_marker.center = screen_pos
        self.robot_marker.angle = math.degrees(pose['yaw'])
        # Gambar titik baru ke garis
        if self.path_line:
            self.path_line.points += [screen_pos[0], screen_pos[1]]
```

```
#
==================================================================
===========
# 5. CLASS APLIKASI UTAMA (Main Logic)
#
==================================================================
===========
class MainApp(App):
    PAN_STEP = 50

    def build(self):
        # Inisialisasi ROS Manager
        self.manager = RosManager(status_callback=self.update_status_label)
        self.nav_goal_coords = None
        self.nav_status_event = None
        self.active_sound = None
        Window.fullscreen = 'auto'

        # --- DESAIN UI (KV Language) ---
        kv_design = """
<Screen>:
    canvas.before:
        Color:
            rgba: 1, 1, 1, 1
        Rectangle:
            pos: self.pos
            size: self.size
<Label>:
    color: 0, 0, 0, 1
<Button>:
    color: 1, 1, 1, 1
    background_color: 0.3, 0.3, 0.3, 1
```

```
    background_normal: ''
<TextInput>:
    foreground_color: 0, 0, 0, 1
    background_color: 0.9, 0.9, 0.9, 1
<ImageButton>:
    background_color: 1, 1, 1, 0
    background_normal: ''
    allow_stretch: True
    keep_ratio: True
    canvas.after:
        Color:
            rgba: 0, 0, 0, 0.3 if self.state == 'down' else 0
        Rectangle:
            pos: self.pos
            size: self.size
<MapControlButton@Button>:
    font_size: '30sp'
    size_hint: (1, 1)
    background_color: 0.2, 0.2, 0.2, 0.5
<WindowToggleBtn@Button>:
    text: '[ ]'
    font_size: '20sp'
    bold: True
    size_hint: None, None
    size: '100dp', '100dp'
    background_color: 0.8, 0, 0, 0.5
    pos_hint: {'top': 1, 'right': 1}
    on_press: app.toggle_window_mode()
<RobotMarker>:
    canvas.before:
        PushMatrix
```

```
        Rotate:
            angle: self.angle
            origin: self.center
    canvas.after:
        PopMatrix
<MapImage>:

<HomeScreen>:
    FloatLayout:
        canvas.before:
            Color:
                rgba: 1, 1, 1, 1
            Rectangle:
                pos: self.pos
                size: self.size
        Image:
            source: 'home.png'
            size_hint: None, None
            width: root.width * 0.58
            height: root.height * 0.58
            pos_hint: {"center_x": 0.5, "center_y": 0.65}
            allow_stretch: True
            keep_ratio: True
        ImageButton:
            source: 'start.png'
            size_hint: None, None
            width: root.width * 0.5
            height: root.height * 0.5
            pos_hint: {"center_x": 0.5, "center_y": 0.20}
            on_press: app.enter_main_menu()
```

```
<NavSelectionScreen>:
    FloatLayout:
        BoxLayout:
            orientation: 'vertical'
            padding: 20
            spacing: 10
            Image:
                source: 'Choose_your_mission.png'
                size_hint_y: 0.2
                allow_stretch: True
                keep_ratio: True
            BoxLayout:
                orientation: 'horizontal'
                spacing: 10
                ScrollView:
                    id: map_scroll
                    BoxLayout:
                        id: nav_map_grid
                        orientation: 'vertical'
                        size_hint_y: None
                        height: self.minimum_height
                        spacing: 20
                        padding: 10
                BoxLayout:
                    id: scroll_container
                    orientation: 'vertical'
                    size_hint_x: None
                    width: '90dp'
                    spacing: 10
                    ImageButton:
                        id: btn_scroll_up
```

```
                source: 'scroll_up.png'
                on_press: app.scroll_map_list_up()
            ImageButton:
                id: btn_scroll_down
                source: 'scroll_down.png'
                on_press: app.scroll_map_list_down()
        ImageButton:
            source: 'go_back.png'
            size_hint_y: None
            height: '150dp'
            size_hint_x: 0.8
            pos_hint: {'center_x': 0.5}
            on_press: root.manager.current = 'main_menu'
    WindowToggleBtn:

<NavigationScreen>:
    name: 'navigation'
    FloatLayout:
        BoxLayout:
            orientation: 'vertical'
            padding: 10
            spacing: 10
            FloatLayout:
                id: map_container
                size_hint: 1, 1
                Scatter:
                    id: scatter_map
                    size_hint: (1, 1)
                    do_rotation: False
                    do_scale: True
                    do_translation: False
```

```
    scale_min: 1.0
    scale_max: 8.0
    auto_bring_to_front: False
    MapImage:
        id: map_viewer
        source: ''
        allow_stretch: True
        keep_ratio: True
        size_hint: (None, None)
        size: self.parent.size
BoxLayout:
    orientation: 'vertical'
    size_hint: (None, None)
    size: ('140dp', '170dp')
    pos_hint: {'x': 0.02, 'y': 0.02}
    spacing: 20
    MapControlButton:
        text: "+"
        on_press: app.zoom_in()
    MapControlButton:
        text: "-"
        on_press: app.zoom_out()
ImageButton:
    id: dpad_up
    source: 'scroll_up.png'
    size_hint: (None, None)
    size: ('100dp', '100dp')
    pos_hint: {'center_x': 0.5, 'top': 0.98}
    on_press: app.pan_map_up()
ImageButton:
    id: dpad_down
```

```
        source: 'scroll_down.png'
        size_hint: (None, None)
        size: ('100dp', '100dp')
        pos_hint: {'center_x': 0.5, 'y': 0.12}
        on_press: app.pan_map_down()
    ImageButton:
        id: dpad_left
        source: 'scroll_left.png'
        size_hint: (None, None)
        size: ('100dp', '100dp')
        pos_hint: {'x': 0.02, 'center_y': 0.5}
        on_press: app.pan_map_left()
    ImageButton:
        id: dpad_right
        source: 'scroll_right.png'
        size_hint: (None, None)
        size: ('100dp', '100dp')
        pos_hint: {'right': 0.98, 'center_y': 0.5}
        on_press: app.pan_map_right()
BoxLayout:
    size_hint_y: None
    height: '60dp'
    orientation: 'horizontal'
    spacing: 10
    Label:
        id: navigation_status_label
        text: 'Status: Pilih titik di peta'
        font_size: '18sp'
    ImageButton:
        id: navigate_button
        source: 'start_navigation.png'
```

```
                size_hint_y: None
                height: '110dp'
                size_hint_x: 1
                pos_hint: {'center_x': 0.5}
                disabled: True
                on_press: app.confirm_navigation_goal()
            ImageButton:
                source: 'go_back.png'
                size_hint_y: None
                height: '110dp'
                size_hint_x: 1
                pos_hint: {'center_x': 0.5}
                on_press: app.exit_navigation_mode()
        WindowToggleBtn:

ScreenManager:
    id: sm
    HomeScreen:
        name: 'home'
    Screen:
        name: 'main_menu'
        FloatLayout:
            BoxLayout:
                orientation: 'vertical'
                padding: [20, 20, 20, 20]
                spacing: 20
                Image:
                    source:'waiter_bot_control_center.png'
                    size_hint_y: None
                    height:'275dp'
                    allow_stretch: True
```

```
            keep_ratio: True
        ImageButton:
            source: 'control_robot.png'
            size_hint_y: None
            height: '220dp'
            size_hint_x: 0.8
            pos_hint: {'center_x': 0.5}
            on_press: app.go_to_controller_mode()
        ImageButton:
            source: 'make_a_map.png'
            size_hint_y: None
            height: '220dp'
            size_hint_x: 0.8
            pos_hint: {'center_x': 0.5}
            # FIX: Panggil fungsi python agar audio jalan
            on_press: app.go_to_pre_mapping_mode()
        ImageButton:
            source: 'do_navigation.png'
            size_hint_y: None
            height: '220dp'
            size_hint_x: 0.8
            pos_hint: {'center_x': 0.5}
            # FIX: Panggil fungsi python agar audio jalan
            on_press: app.go_to_nav_selection_mode()
    WindowToggleBtn:
Screen:
  name: 'pre_mapping'
  FloatLayout:
    BoxLayout:
      orientation: 'vertical'
      padding: [40, 5, 40, 40]
```

```
spacing: 15
Image:
    source: 'name_your_map.png'
    size_hint_y: None
    height: '340dp'
    allow_stretch: True
    keep_ratio: True
TextInput:
    id: map_name_input
    hint_text: 'Contoh: peta_lantai_1'
    font_size: '50sp'
    multiline: False
    size_hint_y: None
    size_hint_x: 0.8
    pos_hint: {'center_x': 0.5}
    height: '120dp'
    padding_y: [30, 0]
Widget:
    size_hint_y: None
    height: '40dp'
ImageButton:
    source: 'start_mapping.png'
    size_hint_y: None
    height: '220dp'
    size_hint_x: 0.8
    pos_hint: {'center_x': 0.5}
    on_press: app.go_to_mapping_mode(map_name_input.text)
    disabled: not map_name_input.text
Widget:
    size_hint_y: None
    height: '20dp'
```

```
            ImageButton:
                source: 'go_back.png'
                size_hint_y: None
                height: '150dp'
                size_hint_x: 0.8
                pos_hint: {'center_x': 0.5}
                on_press: sm.current = 'main_menu'
        WindowToggleBtn:

Screen:
    name: 'controller'
    FloatLayout:
        BoxLayout:
            orientation: 'vertical'
            padding: 40
            spacing: 85
            Image:
                source:'use_controller_move_the_robot.png'
                size_hint_y: None
                height:'550dp'
                allow_stretch: True
                keep_ratio: True
            ImageButton:
                source: 'go_back.png'
                size_hint_y: None
                height: '150dp'
                size_hint_x: 0.8
                pos_hint: {'center_x': 0.5}
                on_press: app.exit_controller_mode()
        WindowToggleBtn:
```

```
Screen:
    name: 'mapping'
    FloatLayout:
        BoxLayout:
            orientation: 'vertical'
            padding: 40
            spacing: 20
            Image:
                source: 'mapping_on_progress.png'
                allow_stretch: True
                keep_ratio: True
                size_hint_y: 0.8
            BoxLayout:
                orientation: 'horizontal'
                spacing: 20
                size_hint_y: None
                height: '120dp'
                ImageButton:
                    source: 'done_save_map.png'
                    size_hint_y: None
                    height: '150dp'
                    allow_stretch: True
                    keep_ratio: True
                    on_press: app.exit_mapping_mode()
                ImageButton:
                    source: 'cancel.png'
                    size_hint_y: None
                    height: '150dp'
                    allow_stretch: True
                    keep_ratio: True
                    on_press: app.cancel_mapping_mode()
```

```
            WindowToggleBtn:

        NavSelectionScreen:
            name: 'nav_selection'
        NavigationScreen:
            name: 'navigation'
"""
        return Builder.load_string(kv_design)

    def toggle_window_mode(self):
        if Window.fullscreen == 'auto':
            Window.fullscreen = False
            Window.maximize()
        else:
            Window.fullscreen = 'auto'

    def _get_map_scatter(self):
        try:
            return self.root.get_screen('navigation').ids.scatter_map
        except Exception:
            return None

    def set_dpad_visibility(self, is_visible):
        try:
            root_screen = self.root.get_screen('navigation')
            opacity_val = 1.0 if is_visible else 0.0
            disabled_val = not is_visible

            root_screen.ids.dpad_up.opacity = opacity_val
            root_screen.ids.dpad_down.opacity = opacity_val
            root_screen.ids.dpad_left.opacity = opacity_val
```

```
        root_screen.ids.dpad_right.opacity = opacity_val

        root_screen.ids.dpad_up.disabled = disabled_val
        root_screen.ids.dpad_down.disabled = disabled_val
        root_screen.ids.dpad_left.disabled = disabled_val
        root_screen.ids.dpad_right.disabled = disabled_val
    except Exception as e:
        print(f"Error setting D-Pad visibility: {e}")

# --- FUNGSI ZOOM & PAN PETA ---
def zoom_in(self):
    scatter = self._get_map_scatter()
    if scatter:
        scatter.scale = min(scatter.scale * 1.2, scatter.scale_max)
        self.set_dpad_visibility(True)

def zoom_out(self):
    scatter = self._get_map_scatter()
    if scatter:
        scatter.scale = max(scatter.scale / 1.2, scatter.scale_min)
        if scatter.scale <= 1.0:
            scatter.scale = 1.0
            self.set_dpad_visibility(False)
            root_screen = self.root.get_screen('navigation')
            scatter.pos = root_screen.ids.map_container.pos

def pan_map_up(self):
    scatter = self._get_map_scatter()
    if scatter: scatter.y -= self.PAN_STEP

def pan_map_down(self):
```

```
    scatter = self._get_map_scatter()
    if scatter: scatter.y += self.PAN_STEP

def pan_map_left(self):
    scatter = self._get_map_scatter()
    if scatter: scatter.x += self.PAN_STEP

def pan_map_right(self):
    scatter = self._get_map_scatter()
    if scatter: scatter.x -= self.PAN_STEP

def scroll_map_list_up(self):
    try:
        scroll = self.root.get_screen('nav_selection').ids.map_scroll
        new_scroll = min(1.0, scroll.scroll_y + 0.1)
        scroll.scroll_y = new_scroll
    except Exception: pass

def scroll_map_list_down(self):
    try:
        scroll = self.root.get_screen('nav_selection').ids.map_scroll
        new_scroll = max(0.0, scroll.scroll_y - 0.1)
        scroll.scroll_y = new_scroll
    except Exception: pass

# --- HITUNG POSISI GOAL (ROS <-> LAYAR) ---
def calculate_ros_goal(self, touch, image_widget):
    screen = self.root.get_screen('navigation')
    if not image_widget.texture or not self.manager.map_metadata: return
    meta = self.manager.map_metadata
    resolution = meta['resolution']
```

```
origin_x = meta['origin'][0]
origin_y = meta['origin'][1]

norm_w, norm_h = image_widget.texture.size
widget_w, widget_h = image_widget.size
if norm_w == 0 or norm_h == 0: return

img_ratio = norm_w / norm_h
widget_ratio = widget_w / widget_h

if widget_ratio > img_ratio:
    scale = widget_h / norm_h
    offset_x = (widget_w - norm_w * scale) / 2.0
    offset_y = 0.0
else:
    scale = widget_w / norm_w
    offset_x = 0.0
    offset_y = (widget_h - norm_h * scale) / 2.0

if scale == 0: return
touch_on_image_x = touch.pos[0] - image_widget.x - offset_x
touch_on_image_y = touch.pos[1] - image_widget.y - offset_y

pixel_x = touch_on_image_x / scale
pixel_y = touch_on_image_y / scale

map_x = (pixel_x * resolution) + origin_x
map_y = (pixel_y * resolution) + origin_y

screen.selected_goal_coords = (map_x, map_y)
screen.ids.navigate_button.disabled = False
```

```
        screen.ids.navigation_status_label.text = f"Goal: ({map_x:.2f}, {map_y:.2f})"

    # --- KIRIM PERINTAH NAVIGASI KE ROBOT ---
    def confirm_navigation_goal(self):
        screen = self.root.get_screen('navigation')
        # Bersihkan garis jalur lama
        screen.clear_path()
        # Mainkan Audio Start Navigation
        self.play_audio('start_navigation.mp3')

        if screen.selected_goal_coords:
            map_x, map_y = screen.selected_goal_coords

            # Kirim ke ROS Manager
            success = self.manager.send_navigation_goal(map_x, map_y)

            if success:
                print(f"INFO: Perintah GOAL ({map_x:.2f}, {map_y:.2f}) Terkirim!")
                screen.ids.navigation_status_label.text = "Status: Robot Bergerak..."
                self.nav_goal_coords = (map_x, map_y)
                if self.nav_status_event:
                    self.nav_status_event.cancel()
                self.nav_status_event =
Clock.schedule_interval(self.check_navigation_status, 0.5)
                screen.ids.navigate_button.disabled = True
            else:
                screen.ids.navigation_status_label.text = "Status: Gagal Kirim Goal"

    def check_navigation_status(self, dt):
        if not self.nav_goal_coords: return False
        current_pose = self.manager.get_robot_pose()
```

```
    if current_pose:
        dx = current_pose['x'] - self.nav_goal_coords[0]
        dy = current_pose['y'] - self.nav_goal_coords[1]
        distance = math.hypot(dx, dy)
        if distance < 0.20:
            print(f"TARGET TERCAPAI (Jarak {distance:.2f}m). Stop.")
            self.finish_navigation_success()
            return False
    return True

def finish_navigation_success(self):
    try:
        screen = self.root.get_screen('navigation')
        map_viewer = screen.ids.map_viewer
        if map_viewer.marker:
            map_viewer.remove_widget(map_viewer.marker)
            map_viewer.marker = None
    except Exception: pass

    if hasattr(self.manager, '_send_stop_command'):
        self.manager._send_stop_command()

    screen = self.root.get_screen('navigation')
    screen.ids.navigation_status_label.text = "Status: Target Tercapai!"
    screen.ids.navigate_button.disabled = True
    self.nav_goal_coords = None

def play_audio(self, file_name):
    """Memainkan audio secara aman (Thread-Safe)."""
    try:
        if os.path.exists(file_name):
```

```
            if self.active_sound:
                self.active_sound.stop()

            self.active_sound = SoundLoader.load(file_name)

            if self.active_sound:
                self.active_sound.play()
                print(f"INFO: Audio '{file_name}' dimainkan.")
            else:
                print(f"WARNING: SoundLoader gagal memuat '{file_name}'.")
        else:
            print(f"WARNING: File audio '{file_name}' TIDAK DITEMUKAN. Abaikan.")
    except Exception as e:
        print(f"ERROR Audio (Ignored): {e}")

# --- FUNGSI PINDAH LAYAR & AUDIO ---
def go_to_controller_mode(self):
    self.play_audio('control_robot.mp3')
    status = self.manager.start_controller()
    self.update_status_label('controller', 'controller_status_label', status)
    self.root.current = 'controller'

def enter_main_menu(self):
    self.play_audio('start.mp3')
    self.root.current = 'main_menu'

def go_to_pre_mapping_mode(self):
    self.play_audio('make_a_map.mp3')
    Clock.schedule_once(lambda dt: setattr(self.root, 'current', 'pre_mapping'), 0.2)

def go_to_nav_selection_mode(self):
```

```
        self.play_audio('do_navigation.mp3')
        Clock.schedule_once(lambda dt: setattr(self.root, 'current', 'nav_selection'), 0.2)

    def exit_controller_mode(self):
        self.manager.stop_controller()
        self.root.current = 'main_menu'

    def go_to_mapping_mode(self, map_name):
        if not map_name.strip(): return
        status = self.manager.start_mapping(map_name)
        self.root.current = 'mapping'
        Clock.schedule_once(lambda dt: self.update_mapping_labels(status, map_name),
0.1)
        self.play_audio('start_mapping.mp3')

    def update_mapping_labels(self, status, map_name):
        screen = self.root.get_screen('mapping')
        if 'mapping_status_label' in screen.ids: screen.ids.mapping_status_label.text =
status
        if 'current_map_name_label' in screen.ids:
screen.ids.current_map_name_label.text = f"Memetakan: {map_name}"

    def exit_mapping_mode(self):
        self.update_status_label('mapping', 'mapping_status_label', 'Menyimpan peta...')
        Clock.schedule_once(self._start_stop_mapping_thread, 0.1)
        self.play_audio('done_save_map.mp3')

    def _start_stop_mapping_thread(self, dt):
        threading.Thread(target=self._thread_safe_stop_mapping, daemon=True).start()

    def _thread_safe_stop_mapping(self):
        self.manager.stop_mapping()
```

```
    Clock.schedule_once(self._go_to_main_menu)

def cancel_mapping_mode(self):
    self.update_status_label('mapping', 'mapping_status_label', 'Membatalkan...')
    Clock.schedule_once(self._start_cancel_mapping_thread, 0.1)

def _start_cancel_mapping_thread(self, dt):
    threading.Thread(target=self._thread_safe_cancel_mapping, daemon=True).start()

def _thread_safe_cancel_mapping(self):
    self.manager.cancel_mapping()
    Clock.schedule_once(self._go_to_main_menu)

@mainthread
def _go_to_main_menu(self, *args):
    self.root.current = 'main_menu'

def exit_navigation_mode(self):
    if self.nav_status_event:
        self.nav_status_event.cancel()
        self.nav_status_event = None
    self.manager.stop_navigation()
    self.root.current = 'main_menu'

def on_stop(self):
    self.manager.shutdown()

@mainthread
def update_status_label(self, screen_name, label_id, new_text):
    if self.root:
        try:
```

```
            screen = self.root.get_screen(screen_name)
            if screen and label_id in screen.ids:
                screen.ids[label_id].text = new_text
        except Exception: pass

# --- LOGIKA TOMBOL NAVIGASI MANUAL VS PRESET ---
def start_navigation_with_map(self, map_name, *args):
    # Audio default 'making_navigation.mp3' (Hardcoded sesuai permintaan)
    self.play_audio('making_navigation.mp3')
    Clock.schedule_once(lambda dt: self._proceed_start_nav(map_name), 0.2)

def _proceed_start_nav(self, map_name):
    self.manager.start_navigation(map_name)
    self.root.current = 'navigation'

    screen = self.root.get_screen('navigation')
    screen.ids.map_viewer.locked = False
    screen.selected_goal_coords = None
    screen.ids.navigate_button.disabled = True
    screen.ids.navigation_status_label.text = "Status: Pilih titik di peta"

def start_preset_navigation(self, point_name, *args):
    target_x, target_y = 0.0, 0.0

    if point_name == 'A':
        target_x, target_y = -14.75, 6.24
        self.play_audio('point_a.mp3')
    elif point_name == 'B':
        target_x, target_y = -27.49, 7.03
        self.play_audio('point_b.mp3')
    elif point_name == 'C':
```

```
            target_x, target_y = -30.93, 3.02
            self.play_audio('point_c.mp3')

        print(f"INFO: Preset Point {point_name} dipilih ({target_x}, {target_y})")

        self.manager.start_navigation('test1')
        screen = self.root.get_screen('navigation')
        screen.pending_preset_target = (target_x, target_y, point_name)
        screen.use_image_marker = True
        self.root.current = 'navigation'

if __name__ == '__main__':
    MainApp().run()
```

Lampiran 3

manager.py

```
#!/usr/bin/env python3
# 1. IMPORT LIBRARY PYTHON & ROS
import subprocess  # Untuk menjalankan perintah terminal (roslaunch, dll)
import time
import os
import signal
import rospkg      # Untuk mencari lokasi paket ROS
import glob
import yaml        # Untuk membaca file metadata peta (.yaml)
import threading
import math

# Coba import modul ROS. Jika gagal (bukan di lingkungan ROS), program tidak crash.
try:
```

```
    import rospy
    import tf
    from tf.transformations import euler_from_quaternion
    from geometry_msgs.msg import Twist, PoseStamped
except ImportError:
    print("PERINGATAN: Pustaka ROS tidak lengkap. Fitur real-time non-aktif.")
    rospy = None
    tf = None

# ==================================================================
# 2. CLASS: ROS POSE LISTENER (Thread Terpisah)
#    Tugas: Mendengarkan posisi robot secara real-time dari TF (/map -> /base_link)
# ==================================================================
class RosPoseListener(threading.Thread):
    def __init__(self):
        super(RosPoseListener, self).__init__()
        self.daemon = True  # Thread mati otomatis jika program utama mati
        self.listener = None
        self.robot_pose = None
        self._stop_event = threading.Event()
        self._run_event = threading.Event()

    def run(self):
        if not rospy or not tf: return

        # Inisialisasi node jika belum ada
        if not rospy.core.is_initialized():
            rospy.init_node('kivy_ros_manager', anonymous=True, disable_signals=True)
```

```
    self.listener = tf.TransformListener()
    rate = rospy.Rate(10.0) # Update 10 kali per detik

    while not self._stop_event.is_set():
        self._run_event.wait() # Tunggu sampai disuruh mulai
        if self._stop_event.is_set(): break

        try:
            # Dapatkan transformasi posisi robot
            (trans, rot) = self.listener.lookupTransform('/map', '/base_link', rospy.Time(0))

            # Konversi orientasi (Quaternion -> Euler/Yaw)
            _, _, yaw = euler_from_quaternion(rot)

            # Simpan data posisi
            self.robot_pose = {'x': trans[0], 'y': trans[1], 'yaw': yaw}

        except (tf.LookupException, tf.ConnectivityException, tf.ExtrapolationException):
            self.robot_pose = None
            continue
        finally:
            rate.sleep()

def start_listening(self):
    self._run_event.set()

def stop_listening(self):
    self._run_event.clear()
    self.robot_pose = None
```

```
    def stop_thread(self):
        self._stop_event.set()
        self._run_event.set()

    def get_pose(self):
        return self.robot_pose

# ==================================================================
# 3. CLASS UTAMA: ROS MANAGER
#    Tugas: Mengelola semua proses ROS (Mapping, Navigasi, Controller)
# ==================================================================
class RosManager:
    def __init__(self, status_callback):
        # Variabel untuk menyimpan proses subprocess (PID)
        self.roscore_process = None
        self.controller_process = None
        self.mapping_process = None
        self.navigation_process = None

        # Flag status
        self.is_controller_running = False
        self.is_mapping_running = False
        self.is_navigation_running = False

        self.status_callback = status_callback
        self.rospack = rospkg.RosPack()

        self.current_map_name = None
```

```
        self.map_metadata = None
        self.pose_listener = None

        # Publisher ROS
        self.cmd_vel_pub = None
        self.goal_pub = None

        # Mulai sistem
        self.start_roscore_if_needed()
        self._init_ros_node()
        print("INFO: RosManager siap.")

    # --- STARTUP SYSTEM ---
    def start_roscore_if_needed(self):
        """Cek apakah roscore sudah jalan. Jika belum, jalankan."""
        try:
            subprocess.check_output(["pidof", "roscore"])
        except subprocess.CalledProcessError:
            print("INFO: roscore belum berjalan, memulai di latar belakang...")
            try:
                self.roscore_process = subprocess.Popen("roscore", preexec_fn=os.setsid,
                                                        stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
                time.sleep(4) # Tunggu roscore siap
            except Exception as e:
                print(f"FATAL: Gagal memulai roscore: {e}")

    def _init_ros_node(self):
        """Inisialisasi Node dan Publisher."""
        if not rospy: return
        try:
```

```
        if not rospy.core.is_initialized():
            rospy.init_node('kivy_ros_manager', anonymous=True, disable_signals=True)

        # Publisher Kecepatan (untuk stop robot)
        self.cmd_vel_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=1)

        # Publisher Goal (untuk kirim tujuan navigasi)
        self.goal_pub = rospy.Publisher('/move_base_simple/goal', PoseStamped,
queue_size=1)

        # Mulai thread listener posisi
        self.pose_listener = RosPoseListener()
        self.pose_listener.start()
        print("INFO: Node ROS, Cmd_vel & Goal Publisher siap.")
    except Exception as e:
        print(f"FATAL: Gagal inisialisasi ROS: {e}")
        self.pose_listener = None
        self.cmd_vel_pub = None
        self.goal_pub = None

# --- NAVIGATION COMMANDS ---
def send_navigation_goal(self, x, y):
    """Mengirim koordinat tujuan langsung ke topic ROS /move_base_simple/goal"""
    if not self.goal_pub:
        print("ERROR: Publisher Goal belum siap (ROS Error)!")
        return False

    try:
        goal = PoseStamped()
        goal.header.frame_id = "map"
        goal.header.stamp = rospy.Time.now()
```

```
        # Set Posisi (Meter)
        goal.pose.position.x = float(x)
        goal.pose.position.y = float(y)
        goal.pose.position.z = 0.0

        # Set Orientasi (W=1.0 artinya netral/lurus)
        goal.pose.orientation.x = 0.0
        goal.pose.orientation.y = 0.0
        goal.pose.orientation.z = 0.0
        goal.pose.orientation.w = 1.0

        self.goal_pub.publish(goal)
        print(f"SUKSES: Goal dikirim ke ROS -> X:{x}, Y:{y}")
        return True
    except Exception as e:
        print(f"ERROR saat kirim goal: {e}")
        return False

# --- PROCESS MANAGEMENT (KILL/STOP) ---
def _stop_process_group(self, process, name):
    """Menghentikan proses dan semua anak prosesnya (Process Group)."""
    if process and process.poll() is None:
        try:
            os.killpg(os.getpgid(process.pid), signal.SIGTERM)
            try:
                process.wait(timeout=0.5)
            except subprocess.TimeoutExpired:
                os.killpg(os.getpgid(process.pid), signal.SIGKILL)
        except (ProcessLookupError, OSError):
            pass
```

```
        return None

    def _send_stop_command(self):
        """Mengirim perintah STOP ke robot agar berhenti bergerak."""
        print("INFO: Mengirim perintah STOP.")

        # Cara 1: Lewat Publisher (Lebih Cepat)
        if rospy and self.cmd_vel_pub:
            stop_msg = Twist()
            for _ in range(10): # Kirim berkali-kali biar yakin
                self.cmd_vel_pub.publish(stop_msg)
                time.sleep(0.01)
        else:
            # Cara 2: Lewat Terminal (Fallback)
            stop_cmd = 'rostopic pub -1 /cmd_vel geometry_msgs/Twist "linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}"'
            subprocess.run(stop_cmd, shell=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

        # Batalkan goal di move_base
        cancel_cmd = 'rostopic pub -1 /move_base/cancel actionlib_msgs/GoalID -- {}'
        subprocess.Popen(cancel_cmd, shell=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

    # --- CONTROLLER MODE ---
    def start_controller(self, launch_file="controller.launch"):
        if not self.is_controller_running:
            command = f"roslaunch my_robot_pkg {launch_file}"
            self.controller_process = subprocess.Popen(command, shell=True, preexec_fn=os.setsid,
                                                       stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
            self.is_controller_running = True
```

```
            return "Status: AKTIF"
        return "Status: Sudah Aktif"

    def stop_controller(self):
        if self.is_controller_running:
            self.controller_process = self._stop_process_group(self.controller_process, "Controller")
            self.is_controller_running = False
            self._send_stop_command()
        return "Status: DIMATIKAN"

    # --- NAVIGATION MODE ---
    def start_navigation(self, map_name):
        if not self.is_navigation_running:
            try:
                self.current_map_name = map_name
                pkg_path = self.rospack.get_path('autonomus_mobile_robot')
                map_file_path = os.path.join(pkg_path, 'maps', f"{map_name}.yaml")

                # Jalankan AMCL & Move Base dengan peta yang dipilih
                command = f"roslaunch autonomus_mobile_robot gui_navigation.launch map_file:={map_file_path}"
                self.navigation_process = subprocess.Popen(command, shell=True, preexec_fn=os.setsid)
                self.is_navigation_running = True

                # Jalankan Controller juga
                self.start_controller("controller.launch")

                if self.pose_listener:
                    time.sleep(5) # Tunggu AMCL siap
                    self.pose_listener.start_listening()
```

```
            return f"Navigasi dengan peta\n'{map_name}' AKTIF"
        except Exception as e:
            print(f"FATAL: Gagal menjalankan navigasi: {e}")
            return f"GAGAL memulai navigasi!\nError: {e}"
    return "Status: Navigasi Sudah Aktif"

def stop_navigation(self):
    if self.is_navigation_running:
        if self.pose_listener:
            self.pose_listener.stop_listening()

        self.stop_controller()
        self.navigation_process = self._stop_process_group(self.navigation_process, "Navigation")
        self.is_navigation_running = False
        self._send_stop_command()

        self.current_map_name = None
        self.map_metadata = None
    return "Status: DIMATIKAN"

# --- MAPPING MODE ---
def start_mapping(self, map_name):
    if not self.is_mapping_running:
        self.current_map_name = map_name
        command = "roslaunch autonomus_mobile_robot mapping.launch"
        try:
            self.mapping_process = subprocess.Popen(command, shell=True, preexec_fn=os.setsid)
            self.is_mapping_running = True
            self.start_controller("mapping_controller.launch")
```

```
            return "Mode Pemetaan AKTIF.\nSilakan gerakkan robot."
        except Exception as e:
            print(f"FATAL: Gagal menjalankan mapping: {e}")
            return f"GAGAL memulai mapping!\nError: {e}"
    return "Status: Mapping Sudah Aktif"

def stop_mapping(self):
    if self.is_mapping_running:
        self._save_map_on_exit()
        self.stop_controller()
        self.mapping_process = self._stop_process_group(self.mapping_process,
"Mapping")
        self.is_mapping_running = False
        self._send_stop_command()
        self.current_map_name = None
    return "Status: DIMATIKAN"

def cancel_mapping(self):
    if self.is_mapping_running:
        self.stop_controller()
        self.mapping_process = self._stop_process_group(self.mapping_process,
"Mapping")
        self.is_mapping_running = False
        self._send_stop_command()
        self.current_map_name = None
    return "Status: DIBATALKAN"

# --- UTILITIES ---
def _save_map_on_exit(self):
    """Menyimpan peta ke file .pgm dan .yaml"""
    if not self.current_map_name: return
    try:
```

```
            pkg_path = self.rospack.get_path('autonomus_mobile_robot')
            map_save_path = os.path.join(pkg_path, 'maps', self.current_map_name)
            command = f"rosrun map_server map_saver -f {map_save_path}"
            subprocess.run(command, shell=True, check=True, timeout=15,
capture_output=True, text=True)
            print("INFO: Peta berhasil disimpan!")
        except Exception as e:
            print(f"ERROR: Gagal menyimpan peta saat keluar: {e}")

    def shutdown(self):
        """Mematikan semua proses saat aplikasi ditutup."""
        print("INFO: Shutdown dipanggil...")
        if self.pose_listener:
            self.pose_listener.stop_thread()
            self.pose_listener.join()

        self.stop_mapping()
        self.stop_navigation()
        self.stop_controller()
        self._send_stop_command()

        if self.roscore_process:
            self._stop_process_group(self.roscore_process, "roscore")

    def get_robot_pose(self):
        if self.pose_listener:
            return self.pose_listener.get_pose()
        return None

    def get_available_maps(self):
        """Mengambil daftar file peta (.yaml) yang ada di folder maps."""
```

```
    try:
        pkg_path = self.rospack.get_path('autonomus_mobile_robot')
        maps_dir = os.path.join(pkg_path, 'maps')
        map_files = glob.glob(os.path.join(maps_dir, '*.yaml'))
        return [os.path.splitext(os.path.basename(f))[0] for f in map_files]
    except Exception:
        return []

def get_map_image_path(self, map_name):
    """Mencari file gambar (.pgm) untuk ditampilkan di UI."""
    try:
        pkg_path = self.rospack.get_path('autonomus_mobile_robot')

        # [LOGIKA KHUSUS] Jika peta bernama 'test1', cari versi yang diedit
        TARGET_EXPERIMENT_MAP = 'test1'
        if map_name == TARGET_EXPERIMENT_MAP:
            pgm_point_path = os.path.join(pkg_path, 'maps', f"{map_name}edited.pgm")
            if os.path.exists(pgm_point_path):
                print (f"INFO: Memuat peta visualisasi pgm untuk {map_name}")
                return pgm_point_path

        path = os.path.join(pkg_path, 'maps', f"{map_name}.pgm")
        return path if os.path.exists(path) else None
    except Exception as e:
        print (f"Error getting map path: {e}")
        return None

def load_map_metadata(self, map_name):
    """Membaca file .yaml untuk mengetahui resolusi dan origin peta."""
    try:
        pkg_path = self.rospack.get_path('autonomus_mobile_robot')
```

```
        with open(os.path.join(pkg_path, 'maps', f"{map_name}.yaml"), 'r') as f:
            self.map_metadata = yaml.safe_load(f)
    except Exception:
        self.map_metadata = None
```

Lampiran 4

PWM_Ramp_Rate_Motor.ino

```
/* Untuk mengganti mode robot secara manual lewat terminal:
  1. Mode Halus (Akselerasi bertahap):
     rostopic pub -1 /set_mode std_msgs/Int8 "data: 1"
  2. Mode Kasar (Respon Instan/Agresif):
     rostopic pub -1 /set_mode std_msgs/Int8 "data: 0"
*/
// --- 1. IMPORT LIBRARY ---
#include <Servo.h>              // Library dasar untuk sinyal PWM/Servo
#include <HB25MotorControl.h>   // Library khusus driver motor HB-25
#include <ros.h>                // Library komunikasi ROS untuk Arduino (rosserial)
#include <geometry_msgs/Twist.h> // Tipe pesan untuk kecepatan (linear & angular)
#include <std_msgs/Int8.h>       // Tipe pesan untuk ganti mode (angka bulat 8-bit)

// --- 2. KONFIGURASI PIN & MOTOR ---
const byte controlPinL = 6;     // Pin PWM Arduino untuk Motor Kiri
const byte controlPinR = 7;     // Pin PWM Arduino untuk Motor Kanan

// Membuat objek kontrol motor
HB25MotorControl motorControlL(controlPinL);
HB25MotorControl motorControlR(controlPinR);

// --- 3. INISIALISASI NODE ROS ---
ros::NodeHandle nh; // Handler utama untuk komunikasi node ROS

// --- 4. VARIABEL KONTROL KECEPATAN ---
```

```
// Target: Kecepatan yang diminta oleh user/navigasi (Goal)
int target_left_speed = 0;
int target_right_speed = 0;

// Current: Kecepatan motor saat ini (Actual)
int current_left_speed = 0;
int current_right_speed = 0;

// Variabel waktu untuk logika 'Smooth' (Non-blocking delay)
unsigned long lastMotorUpdateTime = 0;
const int MOTOR_UPDATE_INTERVAL = 10; // Update kecepatan setiap 10 milidetik

// Langkah kenaikan kecepatan per interval (Makin kecil = Makin halus)
const int SPEED_STEP = 2;

// --- 5. STATUS MODE ---
bool isSmoothMode = true; // Default nyala di Mode Halus (True)

// --- 6. CALLBACK: MENERIMA PERINTAH KECEPATAN (cmd_vel) ---
// Fungsi ini dipanggil otomatis setiap ada data baru di topic /cmd_vel
void twistCallback(const geometry_msgs::Twist& twist_msg) {

  // Konversi float (m/s) ke integer (nilai PWM motor)
  // 300 adalah faktor kalibrasi (sesuaikan dengan kekuatan motor Anda)
  int linear = (int)(twist_msg.linear.x * 300);
  int angular = (int)(twist_msg.angular.z * 300);

  // Rumus Differential Drive (Mencampur maju & belok)
  // Constrain membatasi nilai agar tidak melebihi batas driver (-500 sampai 500)
  target_left_speed = constrain(linear - angular, -500, 500);
  target_right_speed = constrain(linear + angular, -500, 500);
```

```
  // JIKA MODE KASAR (Smooth Mode OFF):
  // Langsung kirim perintah ke motor tanpa menunggu (Respon Instan)
  if (!isSmoothMode) {
    motorControlL.moveAtSpeed(target_left_speed);
    motorControlR.moveAtSpeed(target_right_speed);
  }
}

// --- 7. CALLBACK: GANTI MODE (/set_mode) ---
// Fungsi ini dipanggil jika ada perintah ganti mode
void modeCallback(const std_msgs::Int8& msg) {
  // Jika pesan = 1, aktifkan smooth mode. Jika 0, matikan.
  isSmoothMode = (msg.data == 1);

  // FITUR KEAMANAN:
  // Reset semua kecepatan ke 0 saat mode berganti untuk mencegah sentakan mendadak
  target_left_speed = 0;
  target_right_speed = 0;
  current_left_speed = 0;
  current_right_speed = 0;

  // Hentikan motor fisik
  motorControlL.moveAtSpeed(0);
  motorControlR.moveAtSpeed(0);
}

// --- 8. PENDAFTARAN SUBSCRIBER ---
// Mendaftarkan topic yang akan didengar oleh Arduino
ros::Subscriber<geometry_msgs::Twist> sub_twist("cmd_vel", &twistCallback);
```

```
ros::Subscriber<std_msgs::Int8> sub_mode("/set_mode", &modeCallback);

// --- 9. SETUP (DIJALANKAN SEKALI) ---
void setup() {
  // Set kecepatan komunikasi serial (harus sama dengan konfigurasi di PC)
  nh.getHardware()->setBaud(115200);

  // Mulai Node ROS
  nh.initNode();

  // Aktifkan Subscriber
  nh.subscribe(sub_twist);
  nh.subscribe(sub_mode);

  // Inisialisasi Driver Motor HB-25
  motorControlL.begin();
  motorControlR.begin();
}

// --- 10. LOOP UTAMA (DIJALANKAN TERUS MENERUS) ---
void loop() {
  // Cek apakah ada pesan masuk dari ROS
  nh.spinOnce();

  // JIKA MODE HALUS AKTIF:
  if (isSmoothMode) {
    unsigned long currentTime = millis();

    // Cek apakah sudah waktunya update (setiap 10ms)
    if (currentTime - lastMotorUpdateTime >= MOTOR_UPDATE_INTERVAL) {
      lastMotorUpdateTime = currentTime;
```

```
    // --- LOGIKA RAMPING (AKSELERASI) MOTOR KIRI ---
    if (current_left_speed < target_left_speed) {
      // Jika kecepatan kurang, tambah sedikit demi sedikit
      current_left_speed = min(current_left_speed + SPEED_STEP, target_left_speed);
    }
    else if (current_left_speed > target_left_speed) {
      // Jika kecepatan berlebih, kurangi sedikit demi sedikit
      current_left_speed = max(current_left_speed - SPEED_STEP, target_left_speed);
    }

    // --- LOGIKA RAMPING (AKSELERASI) MOTOR KANAN ---
    if (current_right_speed < target_right_speed) {
      current_right_speed = min(current_right_speed + SPEED_STEP,
target_right_speed);
    }
    else if (current_right_speed > target_right_speed) {
      current_right_speed = max(current_right_speed - SPEED_STEP,
target_right_speed);
    }

    // Kirim kecepatan hasil perhitungan 'halus' ke motor fisik
    motorControlL.moveAtSpeed(current_left_speed);
    motorControlR.moveAtSpeed(current_right_speed);
   }
  }
  // Jika Mode Kasar, motor sudah digerakkan langsung di dalam 'twistCallback'
}
```